



Protection & Access Control

Brief Overview

This note covers **Protection** and was created from a 153-page PDF file. It covers protection mechanisms, access matrices, capability models, RBAC, network firewalls.

Key Points

- Hierarchy of protection rings and privilege levels
 - The **access matrix** defines domain-to-object rights
 - Capability list vs lock-key implementations
 - How **RBAC** enforces least privilege in modern systems
-



Protection Overview

Protection is the mechanism that controls the access of processes to system resources, enforcing policies such as *need-to-know* and *least privilege*.

- Ring (hierarchical) structures allow processes in lower-access rings to invoke procedures in higher-access rings only under tightly controlled rights.
 - Main disadvantage: they cannot fully enforce the need-to-know principle because any object accessible in a domain D_j (with $j < i$) must also be accessible in all higher domains D_i .
-



Access Matrix

The **access matrix** is an abstract model where rows represent **domains** (process groups) and columns represent **objects** (files, devices, etc.).

Entry $access(i, j)$ is the set of rights a process in domain D_i may perform on object O_j .

Example Matrix

	F1	F2	F3	Laser Printer
D1	read, write	read	–	–

D2	read	read, write	read	print
D3	–	–	read, execute	–
D4	–	–	–	write, write

“–” indicates no rights.

Domain Switching

- A process may switch from domain D_i to D_j **iff** the right **switch** is present in $access(i, j)$.
- Example:
 - $D2$ can switch to $D3$ or $D4$ (both have **switch** in row 2).
 - $D1$ can switch to $D2$ (has **switch** in $access(1, 2)$).

Special Rights

Right	Description
Copy (*)	Allows copying a right from one entry to another (e.g., copying <i>read</i> on $F2$ from $D2$ to $D3$).
Owner	Grants the ability to add or delete any right in a column for which the process is owner.
Control	Allows removal of rights from a row (e.g., a process in D_i can delete a right from $access(k, j)$).

- Variants of the copy right: **transfer**, **propagation limitation**, and **simple copy**.
- Rights can be combined or treated as separate privileges (copy, transfer, etc.).

Implementations of the Access Matrix

Technique	Structure	Typical Use	Advantages	Disadvantages
Global Table	List of triples	Simple prototyping	Easy to understand	Large size; poor locality; costly I/O

Access Lists (per object)	For each object, list	When many domains share same objects	Direct lookup for a given object	Search may be long for large lists
Capability Lists (per domain)	For each domain, list (capabilities)	When a process needs fast verification of its own rights	Localized to process; constant-time check	Revocation is difficult
Lock-Key Mechanism	Objects have locks (bit patterns); domains have matching keys	Compromise between lists and capabilities	Simple matching; keys can be transferred	Length of keys affects performance; still requires OS management

Comparison Summary

Aspect	Global Table	Access Lists	Capability Lists	Lock-Key
Memory use	High	Moderate	Moderate	Low-moderate
Search cost	$O(N)$	$O(\text{number of domains per object})$	$O(\text{number of objects per domain})$	$O(1)$ (key match)
Revocation	Easy (delete triple)	Easy (remove entry)	Hard (distributed)	Easy (change lock)
Typical OS	Early prototypes	UNIX (file ACLs)	Mach, some micro-kernels	Some commercial systems

Access Control & Role-Based Access (RBAC)

Access control assigns permissions to users, groups, or roles for objects (files, devices, etc.).

- Traditional UNIX model: owner, group, others with read/write/execute bits.

- **Solaris 10** extends this with **RBAC**:
 - **Privilege** – permission to invoke a specific system call or option.
 - Privileges are assigned to **roles**; users can assume roles to obtain the needed privileges.
 - This implements the principle of least privilege, reducing risks from super-user or set-uid programs.

Figure 13.8 (described): Users → Roles → Privileges → System calls; a user activates a role, the role grants the required privileges for a specific task.

Revocation of Access Rights

Key Questions

- **Immediate vs. delayed** – When does revocation take effect?
- **Selective vs. general** – Does revocation target specific users or all holders?
- **Partial vs. total** – Revoke some rights or the entire set?
- **Temporary vs. permanent** – Can the right be restored later?

Revocation in Access-List Schemes

- Simple: delete the right from the relevant list entry.
- Can be performed **immediately**, and can be **selective**, **partial**, or **temporary**.

Revocation in Capability Schemes

Method	Mechanism
Reacquisition	Periodically delete capabilities; processes must request them again.
Back-pointers	Objects keep a list of all capabilities pointing to them; on revocation, follow pointers and invalidate them (used in MULTICS).
Indirection	Capabilities reference a global table entry; revocation removes the table entry, making the capability illegal.
Keys	Each capability carries a key ; revocation changes the master key for the object, invalidating all previous capabilities.

	Variants allow selective revocation via key lists.
--	--

- All methods require the **owner** (or privileged domain) to perform the revocation operation.

Capability-Based Systems

Hydra Example

- Implements a **fixed set of system-defined rights** (read, write, execute, etc.) plus **user-defined rights**.
- **Auxiliary rights**: rights associated with specific procedures (e.g., a procedure to manipulate an abstract data type).
- **Rights amplification**: a trusted procedure may temporarily gain additional rights (e.g., write) on an object it receives, then restore the original rights on return.
- **Trust model**: procedures declared **trustworthy** may receive amplified rights, but amplification is limited to prevent misuse.
- **Protection of subsystems**: Hydra subsystems run atop a protection kernel; the kernel enforces policies, while subsystems define their own resource policies using the kernel's primitives.

Key Features

- **Typed objects** – objects carry a type; operations on the type are defined as auxiliary rights.
- **Dynamic right adjustment** – rights can change during a procedure call, then revert automatically.
- **Selective amplification** – only procedures explicitly marked trustworthy may receive extra rights, and only for the specific operation invoked.

These mechanisms illustrate how capability-based systems can enforce fine-grained, dynamic protection while supporting modular, trustworthy components.

Cambridge CAP System

CAP (Capability) system provides protection through two distinct kinds of capabilities.

Types of Capabilities

Capability	Description	Rights
Data capability	Directly interpreted by the CAP microcode.	Standard <i>read</i> , <i>write</i> , <i>execute</i> on the storage segment associated with the object.
Software capability	Not interpreted by the microcode; interpreted only by a privileged procedure written by the application programmer.	Rights are granted only when the procedure is invoked; may include rights amplification (temporary read/write) akin to <i>unseal</i> primitives.

Rights Amplification

When the body of a protected procedure executes, a process can **temporarily acquire** additional rights (e.g., write) on the object referenced by a software capability.

The privilege is limited by **type verification**: only capabilities of the specified type may be passed to the procedure.

Protection Model

- **Privileged procedures** act as security kernels for their subsystems; they are the only entities that can interpret software capabilities.
- Unverified, user-defined procedures **cannot** access storage segments or capabilities outside their own environment.
- The CAP machine's microcode enforces a **baseline trust**; higher-level policies are enforced entirely by the protected procedures.

Design Implications

- Subsystem designers must understand **protection principles** rather than rely on a fixed library of procedures.
- The model yields **economies** in policy formulation because capabilities can be combined with procedure-level checks, but it demands careful specification of which procedures are trusted.



Language-Based Protection (Section 13.9)

Language-based protection integrates access-control specifications directly into the programming language, allowing the compiler or runtime to enforce policies.

Why Move Protection into the Language?

- Traditional OS kernels must validate every access, incurring **high overhead**.
- Embedding protection in the language lets designers **declare** required privileges, reducing runtime checks.
- Provides **fine-grained** control based on abstract data types and object semantics.

Compiler-Based Enforcement (13.9.1)

The compiler can **statically verify** that a program respects declared protection requirements.

Advantages

1. **Declarative specification** – protection needs are expressed alongside program code.
2. **OS-independence** – requirements are not tied to a particular operating system.
3. **Separate enforcement** – the compiler, not the subsystem, supplies the enforcement mechanism.
4. **Natural mapping** – privileges correspond to language **data types**, making specifications intuitive.

How It Works (CAP example)

- All hardware references must pass through a **capability**.
- The language runtime supplies **standard protected procedures** that interpret *software capabilities*.
- Programmers declare policies (e.g., “a process may only read a file during this routine”); the compiler generates calls to the appropriate protected procedures.
- The resulting code relies on the underlying CAP microcode for low-level enforcement.

Kernel vs. Compiler Enforcement

Aspect	Kernel-based	Compiler-supported
Security	Strong, because checks are performed at the OS level	Depends on correctness of the compiler, the translator,

	for every access.	and protection of loaded code.
Flexibility	Limited; adding new policies often requires kernel modification.	High; language extensions can introduce new policies without touching the kernel.
Efficiency	Best when hardware directly supports checks (e.g., tagged capabilities).	Can achieve high efficiency by moving checks to compile time; runtime overhead limited to a few kernel calls.

Requirements for Safe Language-Based Protection

- **Immutable capability representation** – processes must not be able to forge or alter capabilities.
- **Trusted runtime components** – the part of the system that interprets software capabilities must be protected (e.g., by the microcode in CAP).
- **Assumption of code integrity** – compiled code must not be modified before or during execution.

Core Constructs (seal/unseal paradigm)

- **Seal** – a primitive that **encapsulates** a data structure, making it inaccessible without the corresponding *unseal* privilege.
- **Unseal** – granted only to code that holds a specific capability, enabling controlled access to the sealed object.

These constructs give programmers a **procedural** way to express protection policies while keeping the underlying mechanism hidden.



Protection in Java (Section 13.9.2)

Java's security model is built on **protection domains**, **stack inspection**, and the **type-safe object model** of the JVM.

Protection Domains

- Each loaded class is assigned to a **protection domain** that encapsulates the permissions granted to that class (e.g., file read, network access).
- Permissions are typically defined in a **configurable policy file** and may depend on the class's origin URL and digital signatures.

Stack Inspection Mechanism

1. When a method requests a protected resource, the JVM performs a **stack walk** starting from the current frame.
2. The walk proceeds **upward** until it encounters:
 - A frame annotated with **doPrivileged** → the check stops and the access is allowed.
 - A frame lacking the required permission → an **AccessControlException** is thrown.
3. The **doPrivileged** block is a static method in AccessController that receives a PrivilegedAction (or PrivilegedExceptionAction) containing the code to be executed with elevated rights.

Blockquote (definition)

Stack inspection is the process of examining the call stack to determine whether any method in the invocation chain lacks the necessary privileges, thereby deciding whether to grant or deny a resource request.

Example Flow (Figure 13.9)

- **gui()** (untrusted applet) calls **get()**.
- **get()** belongs to a **URL-loader** protection domain that has permission to open network connections via a **proxy**.
- Inside **get()**, a **doPrivileged** block performs **open()** on the proxy; the stack inspection finds the **doPrivileged** annotation and **allows** the connection.
- When **gui()** later attempts **open()** directly, no **doPrivileged** frame is found; the inspection fails and an exception is raised.

Type Safety and Object References

- Java disallows raw memory access; programs manipulate **objects only through references**.
- References **cannot be forged**; they are created and checked by the JVM's load-time and run-time verification.
- **Encapsulation** (private fields, package-level access) together with the JVM's safety checks ensures that a class cannot bypass its protection domain.

Summary of Java Protection Features

- **Protection domains** map classes to permission sets.
- **Stack inspection** enforces that only code with explicit privilege (via `doPrivileged`) can perform sensitive operations.
- **Type safety** guarantees that objects are accessed only through well-defined interfaces, preventing arbitrary memory manipulation.
- The combination of these mechanisms provides a **dynamic, fine-grained** security model suitable for executing mutually distrusting code in the same JVM.

Security Problem

Security is the set of mechanisms that protect system resources (data, CPU, memory, disks, networks) from both accidental misuse and intentional attacks.

- **Four protection layers** (building on the protection concepts in earlier sections):
 1. **Physical** – securing machine rooms, terminals, and workstations against burglary or tampering.
 2. **Human** – authentication, social-engineering defenses (phishing, dumpster diving).
 3. **Operating-system** – enforcing access controls, preventing runaway processes, handling stack overflows, etc.
 4. **Network** – protecting data in transit, detecting interception or denial-of-service attacks.

A system is **secure** only when its resources are used and accessed *as intended* under all circumstances. Absolute security is unattainable; the goal is to make attacks sufficiently costly.

Types of Security Violations

Violation	Goal	Typical Impact
Confidentiality breach	Unauthorized reading or theft of data	Identity theft, credit-card fraud
Integrity breach	Unauthorized modification of data	Corrupted source code, altered financial records

Availability breach	Preventing legitimate access	Denial-of-service (DoS), website defacement
Theft of service	Unauthorized use of system resources	Illicit file-serving, bot-net participation
Denial of service	Disruption of service for legitimate users	Network floods, resource exhaustion

Common Attack Techniques

- **Masquerading** – pretending to be a legitimate user or process to bypass authentication.
- **Replay attack** – retransmitting captured valid messages to gain illicit access.
- **Man-in-the-middle (MITM)** – intercepting and possibly altering communication between two parties.
- **Session hijacking** – taking over an active authenticated session.

Figure 14.1 (not shown) illustrated these attacks with sender/receiver diagrams.

Threats & Attack Vectors

Security measures must address threats at each of the four layers:

1. **Physical** – locks, surveillance, controlled entry.
2. **Human** – security awareness training, anti-phishing filters, secure disposal of documents.
3. **OS** – robust protection mechanisms (access matrices, capabilities, RBAC) and hardening against program-level threats.
4. **Network** – firewalls, intrusion-detection systems, encrypted channels.

Weakness at a higher level (e.g., social engineering) can nullify strong OS protections—*the chain is only as strong as its weakest link*.

Program Threats

Program-level threats exploit bugs or deliberately inserted backdoors to subvert the OS protection model.

Trojan Horse

Trojan horse – a seemingly benign program that, when executed, misuses the executing user's privileges.

- Often hidden in search-path directories; the attacker places a malicious executable with a common name (e.g., ls) ahead of the legitimate one.
- Example: adding . (current directory) to the front of \$PATH lets a malicious program in a friend's directory be run unintentionally.
- Variants:
 - **Login emulators** – capture passwords by pretending to be the login prompt.
 - **Spyware** – silently installs a daemon that reports usage data or downloads ads; can act as a covert channel.

Trap Door (Backdoor)

Trap door – a hidden mechanism inserted by a developer that bypasses normal security checks.

- May check for a special user ID or password and grant unrestricted access.
- Can be embedded in compilers or system utilities, making detection extremely difficult because source code appears clean.
- Because modern systems contain millions of lines of code, exhaustive analysis of every component is rarely feasible.

Logic Bomb

Logic bomb – code that remains dormant until a specific condition is met, then executes a malicious payload.

- Trigger conditions can be dates, removal of a user account, or a change in system configuration.
- When activated, the bomb may launch a backdoor daemon, delete files, or otherwise damage the system.

Stack & Buffer Overflow

Stack/Buffer overflow – an attack that writes more data to a fixed-size buffer than it can hold, overwriting adjacent memory (often the return address) and hijacking control flow.

- **Mechanism:**

1. Attacker supplies an oversized input (e.g., command-line argument).
2. The program copies data without bounds checking (strcpy).
3. The overflow overwrites the saved return address on the stack with the address of attacker-controlled shell code.
4. When the function returns, execution jumps to the injected code, typically spawning a root shell.

- **Illustrative C program** (from the transcript):

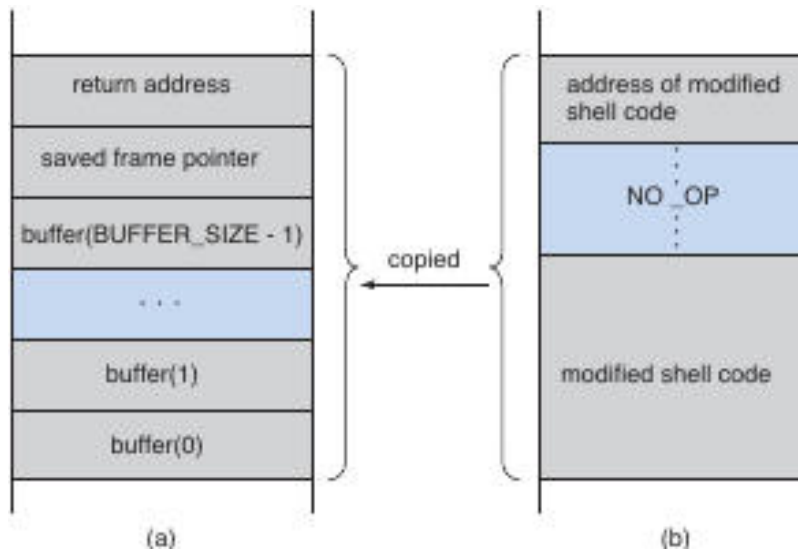
```
#include
#define BUFFER_SIZE 256

int main(int argc, char *argv[]) {
    char buffer[BUFFER_SIZE];
    if (argc < 2) return -1;
    else {
        strcpy(buffer, argv[1]);    // vulnerable copy
        return 0;
    }
}
```

- **Stack layout before overflow** (simplified):

Stack Top →	Parameters	Local variables (buffer)	Saved frame pointer	Return address

- After overflow, the **return address** contains the address of the attacker's shell code.
- **Diagram** – the transcript provides an image that visualizes the overflow:



The diagram shows how the overflow replaces the return address, causing execution of the malicious code.

- **Mitigations:**

- Use safe string functions (strncpy) with explicit length checks.
- Enable hardware **NX (No-Execute)** or **DEP** bits that mark stack pages non-executable (supported on modern AMD/Intel CPUs, Solaris, Linux, Windows XP SP2).
- Compile with stack-canary protections and address-space layout randomization (ASLR).

Viruses

Virus – a self-replicating piece of code that attaches to legitimate programs and spreads to other executables.

- Typically modifies host binaries to include its own code, causing infection of other systems when the host is executed.
- Effects range from file corruption and system crashes to covert data exfiltration.
- Viruses are architecture- and OS-specific; multi-user Unix systems are less susceptible because executable files are often read-only for non-privileged users.
- Countermeasures include:
 - Regular antivirus scanning.
 - Execution permissions that prevent untrusted users from writing executable files.

- Secure boot and code signing to ensure only trusted binaries run.

Summary of Program Threats

Threat	Primary Mechanism	Typical Consequence
Trojan horse	Misleading name/path	Unauthorized privilege use
Trap door	Hidden bypass code	Full system compromise
Logic bomb	Conditional trigger	Delayed malicious action
Stack/Buffer overflow	Overrun fixed buffer	Code execution with attacker's privileges
Virus	Self-replication & infection	Data loss, spread of malware

Understanding these program-level attacks complements the earlier protection models (access matrices, capabilities, RBAC) by highlighting why **correct implementation** and **runtime checks** are essential for a truly secure operating system.

Viruses

Virus – a self-replicating program that inserts its code into other programs or system areas and spreads to additional machines.

Common Transmission Vectors

- **E-mail spam** – attachments or macros that execute when opened.
- **File-sharing services** – users download infected executables.
- **Microsoft Office documents** – Visual Basic for Applications (VBA) macros run automatically under the user's account.

Because macros execute with the privileges of the launching user, they can delete files, modify settings, or e-mail themselves to contacts.

Example VBA Macro (Trojan-style)

```

Sub AutoOpen()
    Dim oFS
    Set oFS = CreateObject("Scripting.FileSystemObject")
    Shell "c:\command.com /k format c:", vbHide
End Sub

```

The macro formats the C: drive when the document is opened.

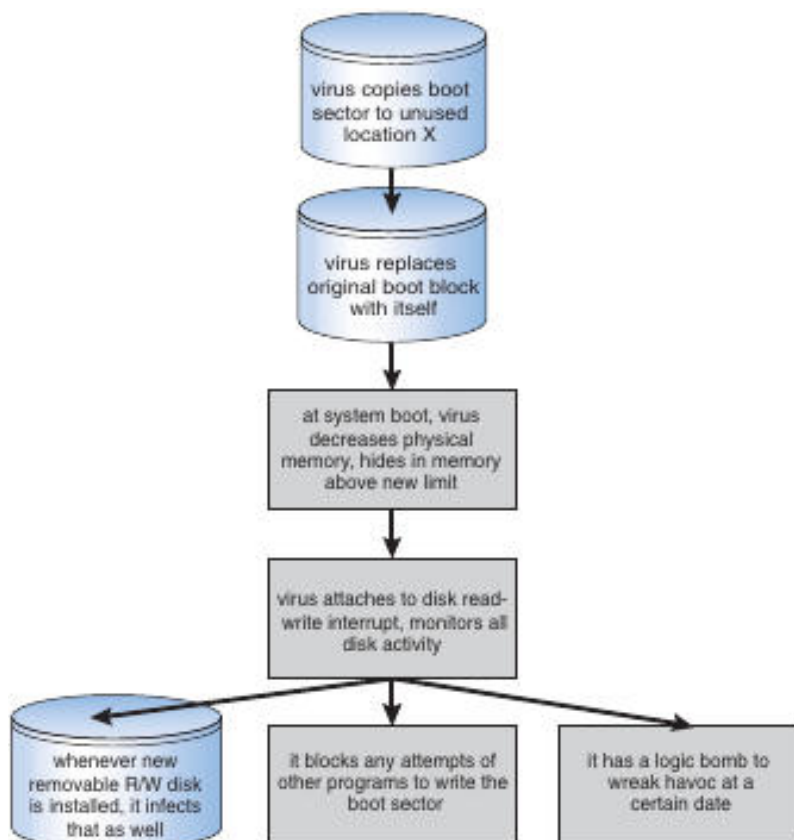
Virus Classification

Category	Typical Target	Key Characteristics
File	Executable files	Appends itself; host program still runs.
Boot	Boot sector of disk	Executes before OS loads; often called <i>memory virus</i> .
Macro	Office documents (Word, Excel, PowerPoint)	Written in high-level languages (e.g., VBA); triggered when the macro runs.
Source-code	Source files	Inserts its code into other source files to propagate.
Polymorphic	Any	Alters its code on each infection to evade signature detection.
Encrypted	Any	Stores encrypted payload; decrypts at run-time to hide signature.
Stealth	Any	Intercepts system calls (e.g., read) to hide infected code from scanners.
Tunneling	System interrupt handlers	Hides in device drivers or interrupt chains.
Multipartite	Both boot sector and files	Infects multiple system components simultaneously.

Armored	Any	Uses heavy obfuscation/compression to resist analysis.
---------	-----	--

Boot-Sector Virus Flow

The diagram below illustrates the life-cycle of a boot-sector virus, from copying the original boot block to hiding in memory and infecting removable media.



The virus copies the original boot sector, replaces it with its own code, then monitors disk activity to infect new media.

Worms

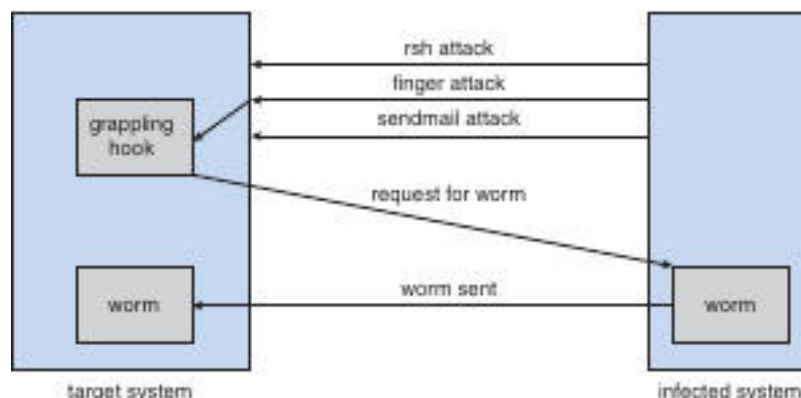
Worm – a self-replicating process that spreads across a network without needing to attach to a host program.

Classic Example: The 1988 Morris Worm

- **Propagation mechanisms**
 1. **Remote shell (rsh) attack** – uses trusted rsh connections without passwords.
 2. **Finger buffer-overflow attack** – sends an oversized string to finger, overwriting the return address to spawn a shell.
 3. **Sendmail exploit** – abuses a debugging option to execute code via crafted mail messages.
- **Password-cracking stage** – tries no password, then simple combinations, followed by a dictionary of 432 common passwords, and finally a full UNIX dictionary.
- **Self-control** – allows every seventh duplicate to continue execution, preventing premature termination of the worm's spread.

Worm Infection Flow

The following flowchart depicts the interaction between an infected host and a target system, showing the three primary attack vectors (rsh, finger, sendmail).



The infected system requests the worm, which is transferred via one of the three attacks and then installed on the target.

Modern Example: Sobig.F (2003)

- Spread by **email**: automatically harvested addresses and sent itself as an attachment.

- Used **randomized subject lines** (“Thank You!”, “Your active ...”) to evade filters.
 - Attempted to download a second-stage payload from a list of 20 servers (servers were shut down before the payload could be retrieved).
-

Port Scanning

Port scanning – a technique that probes a host’s TCP/UDP ports to discover open services and their versions.

- Does **not** exploit a vulnerability directly; it merely gathers information.
- Tools (e.g., **nmap**) can enumerate open ports, identify service banners, and infer operating-system details.
- Because scans are detectable (see Section 14.6.3), attackers often launch them from **zombie** machines to hide their origin.

Typical use case: an attacker discovers that port 25 (SMTP) is running a vulnerable version of sendmail, then proceeds to exploit the identified bug.

Denial-of-Service (DoS)

DoS attack – an attempt to render a service unavailable to legitimate users by exhausting resources.

Two Main Categories

1. **Resource-exhaustion attacks** – consume CPU, memory, or bandwidth (e.g., a Java applet that loops forever).
 2. **Protocol-abuse attacks** – exploit TCP/IP behavior, such as sending a large number of half-open connections (SYN flood) to tie up server sockets.
- **Distributed DoS (DDoS)** amplifies the effect by using many compromised hosts (zombies) to launch coordinated attacks.
 - Some DoS variants target administrative mechanisms, e.g., repeatedly failing login attempts to lock out accounts, or triggering firewall rules that block legitimate traffic.
-

Cryptography as a Security Tool

Cryptography – the science of using secrets (keys) to transform data, providing confidentiality, integrity, and authentication.

Core Concepts

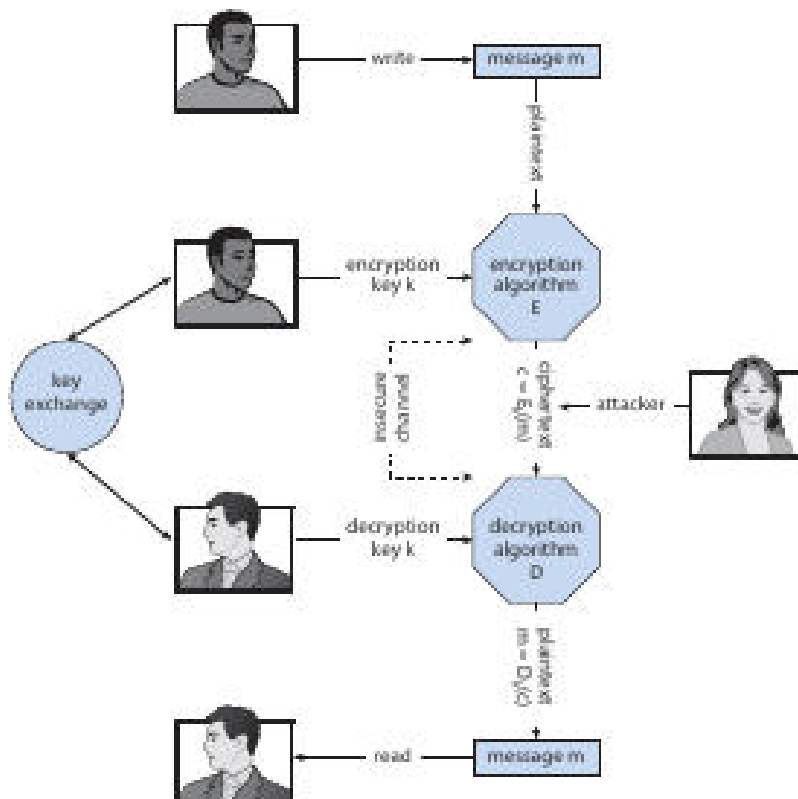
- **Key set \$K\$, message set \$M\$, ciphertext set \$C\$.**
- **Encryption function** $E_k : M \rightarrow C$ generates ciphertexts.
- **Decryption function** $D_k : C \rightarrow M$ recovers the original message.
- Property: $D_k(E_k(m)) = m$ for all $m \in M$ and $k \in K$.

Symmetric Encryption

- Same key k is used for both encryption and decryption.
- Example: **Data Encryption Standard (DES)** – 64-bit block size, 56-bit key, now considered insecure due to exhaustive-search feasibility.
- **Triple DES (3DES)** – applies DES three times (encrypt-decrypt-encrypt) to improve security.

Secure Communication Diagram

The diagram shows two parties exchanging messages over an insecure channel using a shared secret key.



Both parties encrypt with the shared key before transmission; the receiver decrypts with the same key.

Asymmetric Encryption (Brief Mention)

- Uses a **public key** for encryption and a **private key** for decryption (or vice-versa for signatures).
- Enables secure key exchange without prior shared secret, forming the basis for certificates and secure protocols (e.g., TLS).

Summary of Threats & Defenses

Threat	Primary Mechanism	Typical Countermeasure
Virus	Self-replication via files, macros, boot sector	Antivirus signatures, heuristic detection, macro security settings

Worm	Network-wide self-replication exploiting services	Patch vulnerable services, limit unnecessary network daemons, intrusion-detection systems
Port scan	Information gathering	Firewalls with stealth-scan detection, rate-limiting
DoS / DDoS	Resource exhaustion or protocol abuse	Traffic filtering, load balancing, rate-limiting, redundant infrastructure
Cryptographic breach	Key compromise or weak algorithms	Strong key management, use of vetted algorithms (AES, RSA), regular key rotation

These notes build on the earlier [access-matrix](#) and [capability](#) concepts, illustrating why robust protection mechanisms must be complemented by vigilant detection and cryptographic safeguards.

Asymmetric Encryption Basics

Asymmetric encryption (public-key cryptography) uses a *public key* $k_{\{e\}}$ for encryption and a *private key* $k_{\{d\}}$ for decryption.

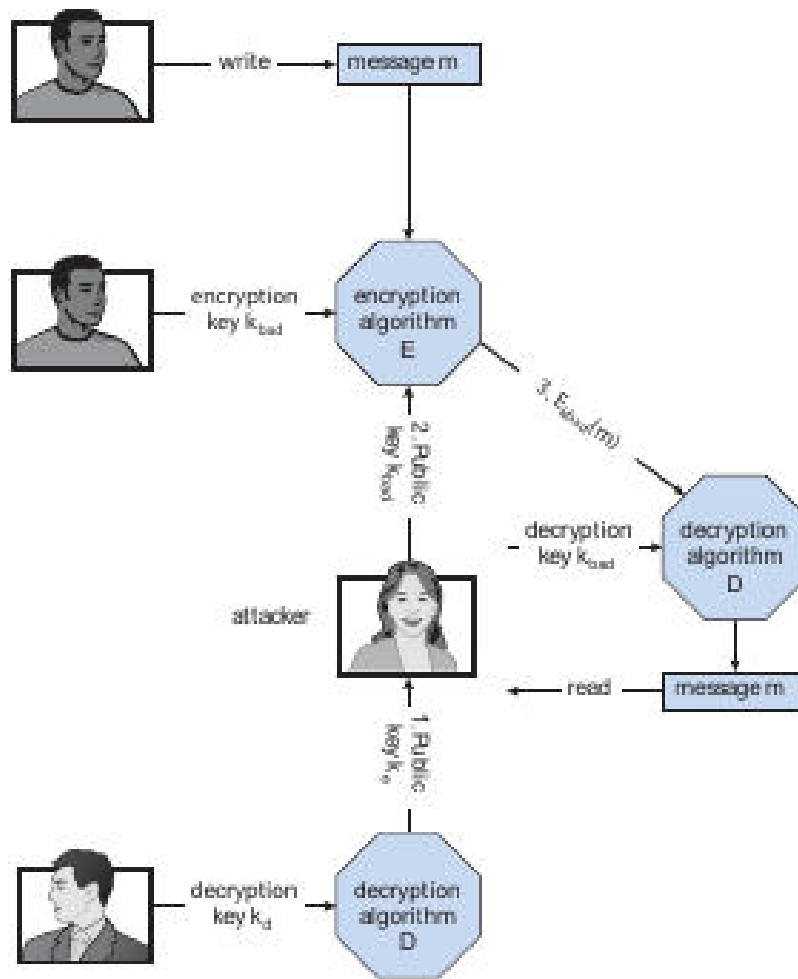
The public key can be widely distributed, while the private key must remain secret.

- **Sender** encrypts a message m with the receiver's public key:

$$c = E_{\{k_{\{e\}}\}}(m)$$
- **Receiver** decrypts with the matching private key:

$$m = D_{\{k_{\{d\}}\}}(c)$$

The exchange of public keys is performed via a [key-exchange](#) process (e.g., certificates, out-of-band distribution).



The diagram illustrates that only the holder of the private key can recover the original message, even if the ciphertext is intercepted.

Symmetric Block Ciphers & Modes

Cipher	Block Size	Key Length(s)	Typical Use
DES	64 bits	56 bits (effective)	Historical, now considered insecure
3DES	64 bits	168 bits (three 56-bit keys)	Legacy systems needing stronger security
AES	128 bits	128, 192, 256 bits	Modern standard (replaced DES)

- **Block ciphers** encrypt fixed-size blocks; to protect longer messages they are combined with **modes of operation** (CBC, CTR, GCM, etc.).
 - **Triple DES** applies DES three times (encrypt-decrypt-encrypt) to increase effective key length.
 - **AES** is compact, efficient, and widely supported in hardware and software.
-

Stream Cipher – RC4

RC4 is a stream cipher that generates a pseudorandom keystream which is XORed with the plaintext.

- Input: a secret key fed into a **pseudo-random bit generator** → produces an (theoretically) infinite keystream.
- Encryption/decryption:

$$\text{ciphertext} = \text{plaintext} \oplus \text{keystream}$$
(XOR yields 0 when bits match, 1 otherwise).

Uses & weaknesses

- Employed in early versions of **WEP** for wireless LANs.
 - Subsequent analyses revealed practical attacks; RC4 is now considered insecure for most applications.
-

RSA Algorithm Details

RSA is the most widely used public-key encryption algorithm; its security relies on the difficulty of factoring large integers.

Key Generation (summary)

1. Choose two large random primes p and q (e.g., 512 bits each).
2. Compute $N = p \times q$ and $\phi(N) = (p-1)(q-1)$.
3. Select a public exponent k_e such that $\gcd(k_e, \phi(N)) = 1$ and $k_e < \phi(N)$.
4. Compute the private exponent k_d satisfying $k_e k_d \equiv 1 \pmod{\phi(N)}$.

Example with small numbers

- $p = 7$, $q = 13 \rightarrow N = 91$, $\phi(N) = 72$
- Choose $k_e = 5$ (coprime to 72).
- Find $k_d = 29$ because $5 \times 29 \equiv 1 \pmod{72}$.

Public key: $(k_e, N) = (5, 91)$

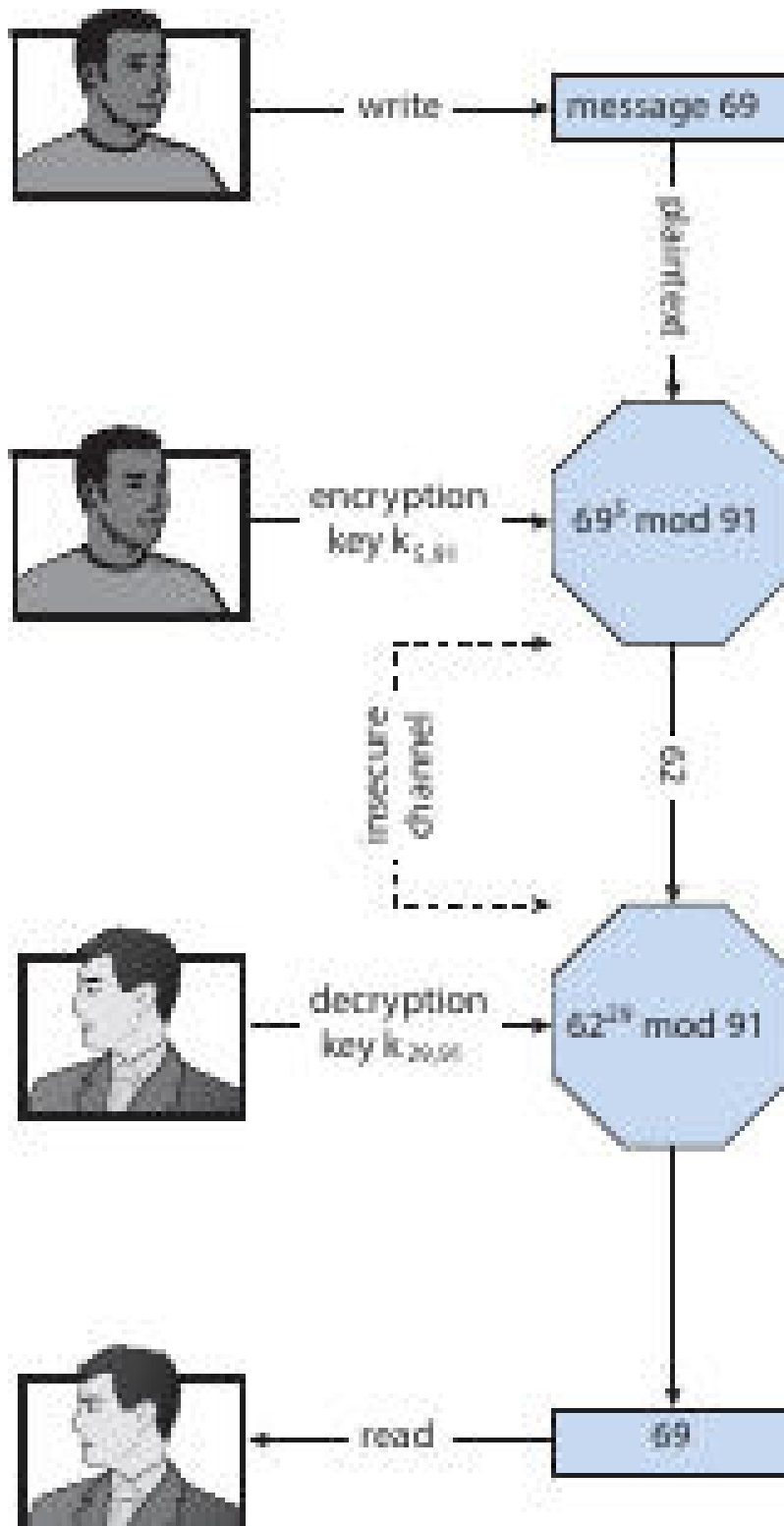
Private key: $(k_d, N) = (29, 91)$

Encryption of message $m=69$:

$$c = m^{k_e} \bmod N = 69^5 \bmod 91 = 62$$

Decryption:

$$m = c^{k_d} \bmod N = 62^{29} \bmod 91 = 69$$



The diagram shows the mathematical steps of RSA encryption and decryption for the small example.

Practical notes

- Public keys are freely published; private keys must be **zealously guarded**.
 - RSA encryption is computationally expensive; it is typically used only for encrypting small data (e.g., session keys) rather than bulk traffic.
-

Authentication Mechanisms

1. Hash Functions

A **hash function** H maps an arbitrary-length message m to a fixed-size digest $H(m)$.

It must be **collision-resistant**: finding $m \neq m'$ with $H(m) = H(m')$ should be infeasible.

Common hashes (now considered weak): **MD5** (128 bits), **SHA-1** (160 bits). Modern systems prefer SHA-256 or stronger.

2. Message-Authentication Codes (MAC)

- A **MAC** is computed as $a = \text{MAC}_k(m)$ using a secret key k .
- Verification: $V_k(m, a) = \text{true}$ only if the verifier possesses the same k .
- Combining a MAC with a collision-resistant hash (e.g., HMAC) yields strong authenticity.

3. Digital Signatures

Component	Role
Private signing key k_s	Generates signature $S_{k_s}(m) = H(m)^{k_s} \bmod N$ (RSA example)
Public verification key k_v	Checks $V_{k_v}(m, S) \iff S^{k_v} \bmod N = H(m)$

- **Non-repudiation**: the signer cannot deny having produced the signature, because only the private key could generate it.
 - RSA signatures reuse the RSA modulus N but reverse the key usage (private for signing, public for verification).
-

Key Distribution Challenges & Solutions

- **Symmetric key distribution** requires a confidential channel for each pair of participants; scaling to many users is impractical.
 - **Asymmetric cryptography** solves this by allowing public keys to be exchanged openly.
 - **Man-in-the-middle (MitM) risk**: an attacker may replace a legitimate public key with a forged one.
 - **Digital certificates** (X.509) bind a public key to an identity, signed by a trusted **Certificate Authority (CA)**.
 - The client verifies the CA's signature using the CA's public verification key.
 - A **web of trust** can be built by chaining certificates from multiple CAs.
-

SSL/TLS Handshake Overview

SSL/TLS provides secure communication over an insecure network by combining asymmetric key exchange with symmetric session encryption.

Simplified handshake steps

1. **ClientHello** – client sends a random nonce n_c and supported cipher suites.
2. **ServerHello** – server replies with its random nonce n_s and its **certificate** $cert_s$ (contains public key $k_{e,s}$).
3. **Certificate verification** – client validates the certificate signature using the CA's public key.
4. **Pre-master secret** – client generates a fresh secret pms (≈ 46 bytes) and encrypts it:
$$c_{pms} = E_{k_{e,s}}(pms)$$

The ciphertext is sent to the server.
5. **Deriving the master secret** – both sides compute
$$ms = \text{PRF}(pms, n_c || n_s)$$

(PRF = pseudo-random function).
6. **Session keys** – from ms derive:
 - Symmetric encryption keys $k_{c \rightarrow s}$, $k_{s \rightarrow c}$
 - MAC keys $k_{\text{mac}, c \rightarrow s}$, $k_{\text{mac}, s \rightarrow c}$
7. **Secure data exchange** – each record is encrypted with the appropriate symmetric key and authenticated with the corresponding MAC.

Security properties achieved

- **Confidentiality** – only the client and server know pms and thus the session keys.
 - **Integrity** – MACs detect any tampering of transmitted records.
 - **Authentication** – server's certificate proves its identity; optional client certificates can authenticate the client.
 - **Forward secrecy** (when using Diffie-Hellman variants) – compromise of long-term private keys does not reveal past session keys.
-



Summary Table – Cryptographic Primitives

Primitive	Goal	Typical Algorithm	Key Type
Encryption (confidentiality)	Hide data	RSA (asymmetric), AES (symmetric)	Public / private or secret
Hashing (integrity)	Detect modification	SHA-256, SHA-3	None (one-way)
MAC (authentication & integrity)	Verify sender & integrity	HMAC-SHA256, CMAC	Secret
Digital Signature (authenticity & non-repudiation)	Prove origin	RSA-DS, ECDSA	Private / public
Key exchange	Establish shared secret	Diffie-Hellman, RSA-key transport	Public / private

These concepts build on the **access-matrix** and **capability** mechanisms discussed earlier: cryptographic keys become the *rights* that protect objects, while authentication ensures that only legitimate *domains* may exercise those rights.



Password Security

Password hashing – a one-way function $f(x)$ that is easy to compute but infeasible to invert, used to store passwords without keeping the clear-text secret.

- **Unix password file** stores only the hash of each user's password.

- When a login attempt occurs, the supplied password is hashed and compared to the stored value.
 - Even if the file is read, the original passwords cannot be recovered.
 - **Weakness** – an attacker can run fast hash routines on a dictionary of common words and compare the results to the stored hashes (dictionary attack).
 - **Salting** – a random value (the *salt*) is concatenated with the password before hashing.
 - Guarantees that identical passwords produce different hashes.
 - Forces an attacker to recompute the dictionary for each distinct salt, greatly increasing the effort required.
 - **Additional protections** – newer Unix systems restrict read-access to the password file to the super-user and store the hash in a separate, protected file.
 - **Password length limits** – classic Unix treats only the first eight characters as significant; modern systems should encourage the full password space.
 - **User guidance** – construct memorable but high-entropy passwords, e.g., use the first letters of a phrase mixed with upper-case, digits, and punctuation: “My mother’s Katherine” → **Mmn.isK!**
 - **Passphrases** – allowing spaces and longer strings further expands the secret space.
-

One-Time Passwords (OTP)

One-time password – a value that is valid for a single authentication event, preventing replay attacks.

Challenge-Response Scheme

1. System generates a random *challenge* ch .
2. User computes the authenticator $H(pw, ch)$, where pw is the shared secret and H is a hash function.
3. The authenticator is sent to the system; the system performs the same computation and verifies equality.
4. For the next login a new ch is generated, producing a different authenticator.

Implementation Variants

Variant	Typical Device	Key Characteristics
---------	----------------	---------------------

Hardware token	Card, key-chain dongle, USB device	Generates $H(pw, ch)$ locally; display shows the OTP.
Software calculator	Smartphone or PC application	Synchronizes with the server to produce the same OTP.
PIN-based generator	Simple numeric keypad (e.g., ATM-style)	Uses a short personal identification number as pw .
Code-book	Printed list of pre-computed OTPs	Each entry is used once and then crossed out.
S/Key	Software or code-book system	Relies on a hash chain; the last entry is the strongest secret.

Two-Factor Authentication (2FA)

- Combines **something you know** (password or PIN) with **something you have** (OTP generator).
- Provides stronger assurance than single-factor authentication because an attacker must obtain both factors.

Biometrics

Biometric authentication – verification based on a user's physiological or behavioral characteristics.

- Palm/hand readers:** Measure temperature maps, finger width, and ridge patterns.
- Fingerprint scanners:** Capture ridge details, convert them to numeric sequences, and store a template for later matching.
- Adaptation:** Scanners can adjust for slight finger-placement variations over time.
- Multi-factor use:** Combining a password with a fingerprint read yields a robust two-factor scheme.
- Future trend:** As costs drop, fingerprint readers are becoming common for everyday computer logins.

Note: Even strong authentication can be subverted if the resulting session is not encrypted; session hijacking remains a risk.

Security Policy

Security policy – a documented set of rules defining what is permissible, required, and prohibited for a given system or organization.

- **Purpose:** Provides a common reference for users and administrators; guides implementation of technical controls.
 - **Typical statements:**
 - “All external-facing applications must undergo code review before deployment.”
 - “Passwords must not be shared.”
 - “Port scans shall be performed every six months.”
 - **Characteristics:**
 - **Living document** – reviewed and updated regularly to remain relevant.
 - **Mapping to controls** – ties policy requirements to concrete security mechanisms (e.g., firewalls, audit logging).
-

Vulnerability Assessment

Vulnerability assessment – systematic examination of a system to discover weaknesses that could be exploited.

Core Activities

- **Penetration testing** – scanning for known vulnerabilities (e.g., outdated software, mis-configured services).
- **Social-engineering tests** – evaluate human susceptibility (phishing, password guessing).
- **Risk assessment** – quantify the likelihood and impact of potential incidents to prioritize remediation.

Typical Checks on a Unix Host

- Weak or default passwords.
- Unauthorized set-uid programs.
- Unexpected executables in system directories.
- Improper permissions on sensitive files (e.g., /etc/passwd, device files).
- Dangerous entries in the executable search path.
- Missing checksums on system binaries.
- Hidden network daemons.
- Scans are usually run during low-load periods; they may be performed on production or on a replica system to avoid disruption.



Intrusion Detection & Prevention

IDS (Intrusion-Detection System) – monitors activity and alerts when suspicious behavior is observed.

IDP (Intrusion-Prevention System) – extends IDS by actively blocking detected attacks.

Detection Approaches

Approach	Basis	Example
Signature-based	Known patterns of malicious activity (signatures).	Scanning packets for the string “/etc/passwd”.
Anomaly-based	Deviations from a learned “normal” baseline.	Unusual system-call sequences for a daemon.

- **Signature-based** excels at known threats but cannot detect zero-day attacks.
- **Anomaly-based** can uncover novel attacks but may generate false positives if the baseline is inaccurate.

False-Alarm Illustration (Bayesian estimate)

- Intrusion probability $P(I) = 0.00002$ (2 intrusions per 10^6 events).
- True-positive rate $P(A/I) = 0.8$.

- False-positive rate $P(A/-I) = 0.0001$.
- Resulting $P(I/A) \approx 0.14$: only ~1 in 7 alerts corresponds to a real intrusion, highlighting the “Christmas-tree effect.”
- Effective IDS design strives for **very low false-alarm rates** while maintaining high detection probability.

Virus Protection

Antivirus software – tools that detect, quarantine, and remove malicious code.

Detection Techniques

Technique	How it Works
Signature scanning	Matches known byte patterns of viruses.
Heuristic/family detection	Looks for characteristic behaviors or code structures common to a virus family.
Decompression	Unpacks compressed malware before scanning.
Behavioral analysis (sandboxing)	Executes program in a controlled environment and observes for malicious actions.
Process anomaly detection	Flags suspicious operations, e.g., a non-compiler writing to executable files.

Limitations & Countermeasures

- **Polymorphic viruses** modify their code to evade signature detection → heuristic and sandbox approaches become essential.
- **Prevention:**
 - Acquire software from trusted sources; avoid pirated copies.
 - Disable macro execution or use formats without macro capability (e.g., RTF).
 - Restrict execution permissions on newly downloaded files.

Integrity-Checking Example – Tripwire

- **Purpose:** Detect unauthorized changes to critical files and directories.

- **Operation:**
 1. Initial run creates a database of file attributes (inode metadata, cryptographic hash).
 2. Subsequent runs recompute attributes and compare against the stored signatures.
 3. Reports added, deleted, or altered items.
 - **Configuration** – a `tw.config` file selects which paths and which attributes to monitor (e.g., permissions vs. access times).
 - **Protection** – the Tripwire database and binaries must reside on a tamper-resistant medium (write-protected disk) to prevent an attacker from suppressing alerts.
 - **Limitation** – cannot distinguish between legitimate updates (e.g., log file growth) and malicious modifications without additional policy rules.
-

Auditing, Accounting, and Logging

Audit logging – recording of system-level events (system calls, authentication attempts, privilege escalations) for later analysis.

- **Uses:**
 - Detect unauthorized access or policy violations.
 - Provide forensic evidence after a breach.
 - Identify performance anomalies that may indicate security problems.
 - **Accounting** – aggregates resource usage per user or process; can reveal abnormal consumption patterns.
 - **Historical note:** The first widely reported UNIX break-in was uncovered by examining accounting logs for unexpected activity.
-

Firewalling to Protect Systems and Networks

Firewall – a network device (software or hardware) that enforces a security boundary between trusted and untrusted zones.

- **Functions:**
 - **Packet filtering** – permit or deny traffic based on source/destination IP, ports, and protocol.

- **Stateful inspection** – track connection state to allow only legitimate return traffic.
- **Logging** – record connection attempts for audit and intrusion-detection purposes.
- **Application-level gateways** – proxy specific services (e.g., HTTP) to enforce additional checks.
- **Typical policy example:** Allow only HTTP (port 80) from the Internet to a web server inside the firewall; block all other inbound ports.
- **Security perspective:** Firewalls embody the “defense-in-depth” principle by adding a network-level layer that complements host-based protections (access control, IDS, etc.).

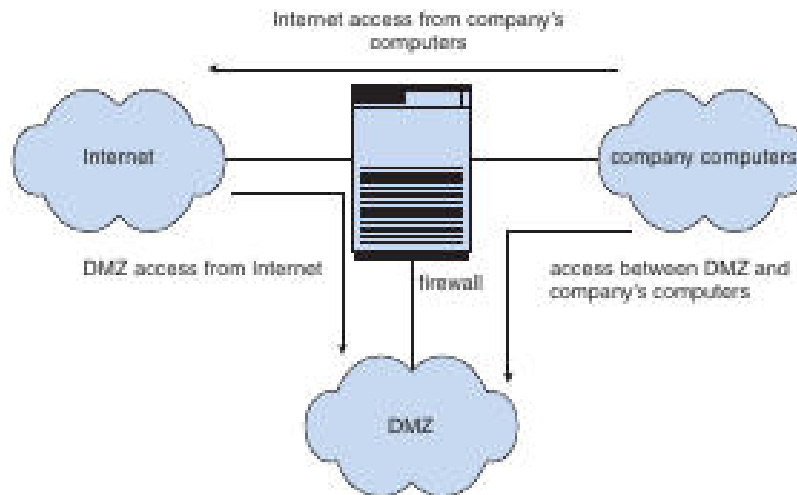


Firewalling to Protect Systems and Networks

Firewall – a network device (hardware or software) that mediates traffic between distinct security domains, enforcing a set of rules that permit or deny connections.

Network Zones & the Demilitarized Zone (DMZ)

- **Typical three-domain layout**
 1. **Internet** – untrusted, semitrusted, or semisecure network.
 2. **DMZ** – a separate zone that hosts services (e.g., web, mail) exposed to the Internet.
 3. **Corporate LAN** – the internal, trusted network of company computers.
- The firewall sits between each pair of domains, controlling the direction of traffic.



The diagram shows that the Internet may reach the DMZ, the corporate LAN can reach both the DMZ and the Internet, and the firewall enforces all inter-domain traffic.

- **Allowed flows**
 - Internet → DMZ (e.g., inbound HTTP/HTTPS).
 - Corporate LAN ↔ DMZ (e.g., a web server in the DMZ querying an internal database).
 - Corporate LAN ↔ Internet (usually outbound only).
- **Blocked flows**
 - Direct Internet → corporate LAN.
 - DMZ → corporate LAN (unless explicitly permitted).
- **Security benefit** – even if a DMZ host is compromised, the attacker cannot directly reach internal machines because the firewall blocks that path.

Types of Firewalls and Their Trade-offs

Type	Typical Scope	Strengths	Weaknesses
Packet-filtering (network-layer)	IP address, port, protocol	Simple, fast, low overhead	No awareness of application semantics; vulnerable to protocol-tunneling attacks
Stateful inspection	Tracks connection state (e.g.,	Blocks illegitimate half-open	Still limited to transport-layer

	SYN-ACK-FIN)	connections; more context than plain filter	information
Application-level proxy	Understands specific protocols (SMTP, HTTP, etc.)	Can inspect payload, enforce protocol rules, hide internal addresses	Higher latency; must be maintained per protocol
Personal host-based firewall	Limits traffic to/from a single host	Protects a workstation from Trojan-horse network activity	Does not protect the wider network
XML / protocol-specific firewalls	Parses and validates a single protocol (e.g., XML)	Detects malformed or disallowed constructs in that protocol	Only useful for the targeted protocol
System-call-filtering firewall (e.g., Solaris “least-privilege” feature)	Intercepts system calls made by processes	Can block dangerous calls (e.g., execve, mknod) per process	Requires kernel support; may impact performance

Limitations & Attack Vectors Against Firewalls

- **Payload-based attacks** – a buffer-overflow exploit embedded in an allowed HTTP request bypasses the firewall because the firewall only checks the *connection*, not the *content*.
- **Spoofing** – if a rule permits traffic from a given IP address, an attacker can forge that address and gain unauthorized access.
- **Denial-of-Service (DoS) on the firewall** – overwhelming the firewall with traffic can render it unable to enforce rules, effectively opening the protected zones.
- **Compromise of the firewall itself** – a poorly secured firewall becomes a single point of failure; attackers can subvert its rule set or use it as a pivot.



Computer-Security Classifications (Orange Book)

TCB (Trusted Computer Base) – the set of hardware, firmware, and software components that enforce a system’s security policy.

The U.S. Department of Defense **Trusted Computer System Evaluation Criteria** (Orange Book) defines four divisions (D → A) with increasing assurance requirements.

Division	Primary Guarantees	Example Systems
D – <i>Minimal Protection</i>	No specific security features; may lack authentication.	MS-DOS, Windows 3.1
C – <i>Discretionary Security</i>	User-level access control, audit trails. Sub-levels: C1, C2.	Most UNIX variants (C1); C2 adds individual-level access control, selective auditing, and TCB self-protection.
B – <i>Mandatory Protection</i>	Mandatory Access Control (MAC) with sensitivity labels; stronger TCB verification. Sub-levels: B1, B2, B3.	Secure UNIX variants (B1); B2 adds labeling of devices and covert-channel analysis; B3 adds ACLs and robust event monitoring.
A – <i>Verified Design</i>	Formal specification and verification of the TCB; highest assurance.	Systems built in a trusted facility with formal methods; rarely deployed commercially.

Highlights of Each Division

- **C1** – provides discretionary controls (owner-defined ACLs) and basic auditing.
- **C2** – refines C1 with:
 - *Individual* access rights (per-user file permissions).
 - *Selective* auditing based on user identity.
 - Protection of the TCB code from modification.
- **B1** – adds **mandatory** sensitivity labeling (e.g., *confidential*, *secret*) and enforces MAC.
- **B2** – extends labeling to *physical devices* and adds covert-channel analysis.
- **B3** – introduces full **access-control lists** for users/groups, and a mechanism that can *terminate* a security-policy violation in a minimally disruptive way.
- **A** – employs **formal verification** to prove that the TCB correctly implements the policy.

Key point – The Orange Book classifies *how* a system enforces security, not *what* policy it enforces. Policy definition remains the responsibility of the system owner.

Windows 7 Security Model

Protection Domain – in Windows 7, each process runs within a *subject* that carries an **access token** describing the subject's identity and privileges.

Access Tokens & Security Identifiers (SIDs)

- When a user logs on, Windows 7 creates an **access token** containing:
 - **User SID** – unique identifier for the account.
 - **Group SIDs** – identifiers for all groups the user belongs to.
 - **Privilege list** – e.g., SeBackupPrivilege, SeShutdownPrivilege.
 - **Integrity label** – one of untrusted, low, medium, high, system.
- Every thread/process inherits a copy of the token; the kernel checks the token's SIDs and privileges against an object's **security descriptor** whenever an operation is attempted.

Security Descriptors

- **Owner SID** – who can change the descriptor.
- **Group SID** – default group for inherited permissions.
- **Discretionary ACL (DACL)** – list of **Access-Allowed** or **Access-Denied** entries, each pairing an SID with an **access mask** (e.g., ReadData, WriteData, Execute).
- **System ACL (SACL)** – specifies which operations generate audit events.

Example DACL (textual)

```
Owner:  S-1-5-21-...-1001   (avi)
DACL:
  avi           – FullControl
  GROUP\cs      – Read, Write
  cliff         – NoAccess
SACL:
  Audit: Write – Success, Failure
```

Mandatory Integrity Control (MIC)

- **Integrity labels** enforce a *no-write-up* rule: a subject may not write to an object whose integrity level is higher than its own.
- **Read/execute** are permitted unless explicitly blocked.
- Objects without an explicit label default to **medium**; non-admin users receive a **medium** label, while elevated processes receive **high**.

Integrity Level	Typical Subjects
Untrusted	Internet-facing sandboxed processes
Low	Limited-privilege services
Medium	Standard interactive users
High	Elevated admin processes (UAC “Run as administrator”)
System	Core OS services

- **MIC access-mask bits**: NoWriteUp, NoExecuteUp, NoReadUp – automatically enforced by the kernel.

User Account Control (UAC)

- An administrator’s account possesses **two tokens**:
 - **Standard token** – medium integrity, admin group disabled.
 - **Elevated token** – high integrity, admin group enabled.
- When a program requests elevation (runas), the system switches to the elevated token after user consent.

Auditing & Logging

- **Audit policies** are expressed in the SACL of a security descriptor.
- Windows 7 can audit:
 - **Logon successes/failures** (detect brute-force attempts).
 - **File accesses** (identify reads/writes of sensitive files).
 - **Executable launches** (track potential malware).
- Audited events are recorded in the **Security Event Log**, which can be forwarded to a SIEM for correlation.



Comparison of Security Mechanisms (Across Platforms)

Mechanism	Primary Layer	Example Implementation	Typical Use
Access Matrix (earlier sections)	OS kernel	Global table, access lists, capability lists	Fine-grained permission checking
RBAC / Privileges (Solaris, Windows)	OS + application	Roles → privileges; Windows privileges in access token	Enforce least-privilege for admin tasks
Firewall	Network	Packet filter, stateful inspection, application proxy	Separate trust domains (Internet ↔ DMZ ↔ LAN)
Mandatory Integrity Control	OS (Windows)	Integrity labels, NoWriteUp rule	Prevent low-integrity code from contaminating higher-integrity objects
TCB / Orange-Book Classification	System design	B2-level UNIX, C2-level Windows NT	Provide assurance levels for whole system
Cryptographic primitives	Communication & storage	TLS handshake, RSA, AES, HMAC	Confidentiality, integrity, authentication

Key Takeaways (no concluding remarks per instructions)

- Firewalls enforce **domain boundaries**; a DMZ isolates externally reachable services while protecting the internal LAN.
- Different firewall architectures trade **performance** for **deep inspection**; choose based on the required threat model.
- The Orange Book's **division hierarchy** (D → A) quantifies how rigorously a system enforces its security policy, from discretionary controls (C) to formally verified designs (A).
- Windows 7's security hinges on **access tokens**, **SIDs**, and **integrity labels**, combining discretionary ACLs with mandatory integrity control to limit the impact of compromised low-privilege processes.

- Auditing, both at the OS (SACL) and network (IDS/IPS) levels, is essential for detecting policy violations and supporting forensic analysis.

Linux History

Linux is a UNIX-like operating system that began in 1991 as a small, self-contained kernel for the Intel80386.

Milestone	Release date	Key features
0.01	May 1991	First public kernel; ran only on 80386 PCs; minimal device-driver support; Minix file system; basic virtual memory with copy-on-write.
1.0	14 Mar 1994	Added TCP/IP networking (BSD-compatible sockets), Ethernet/PPP/SLIP drivers, expanded file-system support (beyond Minix), paging to swap, early SCSI support.
1.2	Mar 1995	Wider hardware range (PCI bus, virtual 8086 mode for DOS emulation), loadable kernel modules, IP accounting/firewalling, first PC-only stable release.
2.0	Jun 1996	Multi-architecture support (including 64-bit), SMP (symmetric multiprocessing), unified file-system cache, writable memory-mapped regions, internal threads, automatic module loading,

		POSIX-compatible real-time scheduling.
2.2	1999	Enhanced networking (routing, TCP window scaling, selective acknowledgments), NFS kernel-mode daemon, improved SMP scalability, support for Alpha, MIPS, SPARC.
2.4 / 2.6	2001-2004	Journaling file systems, advanced memory-management, O(1) scheduler → preemptive kernel, extensive SMP optimizations.
3.0	Jul 2011	Marked 20-year anniversary; Completely Fair Scheduler (CFS) , improved virtualization, new page-fault handling, refined memory management.

As discussed earlier, the evolution of the kernel reflects the need for robust **access-matrix**-style permission checks and **capability**-based extensions as hardware and networking grew more complex.

Linux System Components

A full Linux system consists of three tightly coupled layers: **kernel**, **system libraries**, and **system utilities**.

Layer	Role	Execution mode
-------	------	----------------

Kernel	Manages processes, memory, devices, file systems, networking; implements system calls.	Kernel mode (full hardware privileges).
System libraries (e.g., libc)	Provide the user-visible API; wrap system calls, implement higher-level functions (buffered I/O, string handling).	User mode (access via library calls).
System utilities	Daemons (e.g., sshd, cron) and command-line tools (ls, cp).	User mode (daemons may run with elevated privileges as needed).

The kernel's monolithic design (see § 15.2) keeps all core services in a single address space, avoiding context-switch overhead—an idea echoed in the **access-matrix** model where rights are checked centrally.

Kernel Modules

Loadable kernel modules (LKMs) are object files that can be inserted into or removed from the running kernel without a reboot.

Why LKMs matter

- **Rapid development** – developers can test a new driver by loading it directly, skipping full kernel rebuilds.
- **Hardware flexibility** – a base kernel can stay minimal; drivers for optional devices are loaded on demand.
- **GPL compliance** – because the kernel is GPL-v2, any module linked into it must also be GPL-compatible, ensuring source availability for the whole system.

Module-management architecture

Component	Function
Module-management system	Allocates kernel memory, tracks loaded modules.

Loader / unloader (user-mode utilities)	Initiates insertion (insmod) or removal (rmmod).
Driver-registration subsystem	Announces new drivers to the kernel (e.g., register_chrdev).
Conflict-resolution mechanism	Prevents multiple modules from claiming the same hardware resources.

Loading workflow (simplified)

1. **Resolve symbols** – the module’s undefined symbols are matched against the kernel’s exported symbol table.
2. **Reserve memory** – the kernel allocates a contiguous virtual-memory region for the module.
3. **Relocate code** – the loader rewrites addresses in the module to the allocated base.
4. **Copy into kernel** – the module image is copied, and the kernel updates its internal tables.
5. **Initialize** – the module’s entry point (init_module) runs; registration of devices, filesystems, or protocols occurs.

*If any symbol cannot be resolved, loading aborts and the module is rejected—mirroring the **access-matrix** principle that a domain may act only on objects for which it possesses explicit rights.*

Linux Distributions

A **distribution** bundles the Linux kernel with a coherent set of libraries, utilities, package-management tools, and often a default desktop environment.

Early vs. modern distributions

Distribution	Era	Package management	Notable traits
SLS (Softlanding Linux System)	Early 1990s	Manual unpacking; no dependency tracking	First complete Linux collection, but poor management.

Slackware	1993-present	pkgtool (simple)	Stays close to upstream sources; emphasizes stability.
Debian	1993-present	dpkg / apt (dependency-aware)	Community-driven; strong free-software stance.
Red Hat / Fedora	Mid-1990s-present	rpm / yum / dnf	Commercial support (Red Hat), fast-moving upstream (Fedora).
Ubuntu (based on Debian)	2004-present	apt with user-friendly tools	Popular desktop/server distro; regular releases.
Arch, Gentoo, openSUSE	2000s-present	pacman, portage, zypper	Targeted at advanced users or specific use-cases.

- **Compatibility:** Most distributions understand the **RPM** or **DEB** binary formats, allowing cross-distribution software installation when the package manager is present.
- **Package-tracking database** records installed files, versions, and dependencies, enabling safe upgrades and clean removals—an evolution from the manual unpacking of early systems.

Linux Licensing

The Linux kernel is released under **GNU General Public License version 2 (GPL-v2)**.

- **Freedom to copy, modify, and redistribute** is granted, provided that **source code** for the entire derived work is also made available.
- **No “public-domain” claim:** authors retain copyright; the GPL merely permits wide reuse under its conditions.
- **Implication for modules:** any kernel module linked with the GPL-licensed kernel must itself be GPL-compatible; proprietary binary-only extensions cannot be distributed as part of the kernel.

- **Distribution requirement:** when binaries are shipped, the accompanying source (or a written offer to provide it) must be offered at no more than a reasonable cost.

Design Principles

Linux follows a classic **UNIX** philosophy while adapting to modern hardware and usage scenarios.

Principle	Description	Example in Linux
Preemptive multitasking	The scheduler can interrupt a running process to switch context.	O(1) scheduler → CFS (Completely Fair Scheduler).
Multi-user	Multiple users share the same system, each with isolated resources.	UID/GID, per-process credentials, access-matrix-style permission bits.
Portability	Runs on many CPU families (x86, ARM, PowerPC, etc.).	Kernel version 2.0 introduced multi-architecture support.
Monolithic kernel	All core services (scheduler, memory manager, drivers, networking) reside in one address space for performance.	No micro-kernel message passing; direct function calls.
Loadable modules	Extend functionality without rebuilding the kernel.	modprobe, insmod, rmmod.
POSIX compliance	Provides standardized interfaces for portability of applications.	pthread library, open(), fork(), signals.
Standardization (FHS)	Filesystem Hierarchy Standard defines where configuration files, binaries,	/etc for configs, /usr/bin for user binaries, /var for variable data.

	libraries, and runtime data reside.	
--	-------------------------------------	--

These principles underpin the security mechanisms explored earlier: discretionary access control via file permission bits, mandatory integrity control (e.g., SELinux), and the use of *capabilities* for fine-grained rights.

Components of a Linux System (Recap)

Component	Example	Role
Kernel	vmlinuz	Core resource manager; enforces protection policies.
System libraries	glibc (libc.so)	User-mode interface to kernel services; also provide higher-level utilities (string handling, I/O buffering).
System utilities	sshd, cron, bash	Daemons and command-line tools that perform administration, networking, and user interaction.
Device drivers	e1000 (Intel NIC), ext4 (filesystem)	Often built as LKMs; interact directly with hardware.
Package manager	apt, dnf, pacman	Handles installation, upgrade, and removal of software bundles.

Driver Registration (within Kernel Modules)

When a module provides a driver, it must *register* with the kernel so that the appropriate subsystem can locate it.

- **Registration call** (e.g., `register_chrdev` for character devices) adds the driver's entry to the kernel's internal tables.
- The kernel then routes matching device requests (open, read, write) to the newly registered driver.
- Unregistering (`unregister_chrdev`) removes the entry, preventing further use and allowing safe module unloading.

*This mirrors the **access-matrix** notion of adding a new row (domain) or column (object) at runtime, with the kernel enforcing that only authorized code may modify the matrix.*

Kernel Modules & Dynamic Drivers

Kernel module – a piece of code that can be loaded into the running kernel to add new functionality (e.g., drivers, filesystems, protocol stacks).

- When a module is **loaded**, the kernel:
 1. Allocates an isolated region of kernel memory.
 2. Calls the module's **initialisation (startup)** routine.
 3. The routine registers the services the module provides (device nodes, VFS callbacks, sysctl entries, etc.).
- When a module is **unloaded**, the kernel invokes its **cleanup** routine, which removes the registrations.

Types of Registerable Functionality

Category	What it Provides	Example
Device drivers	Access to character devices (terminals, mice), block devices (disk drives), or other interfaces	tty driver, SATA controller
File systems	Implements VFS hooks; can be disk-based, network-based, or virtual	ext4, NFS, /proc
Network protocols	Full protocol implementation or packet-filtering hooks	TCP stack, firewall module

Binary formats	New executable file type recogniser	ELF loader extension
Sysctl entries	Dynamically configurable kernel parameters	/proc/sys/net/ipv4/ip_forw

The kernel maintains **dynamic tables** for each class; modules add or remove entries at load/unload time.

Conflict-Resolution for Hardware Resources

Resource reservation – a kernel mechanism that tracks allocation of limited hardware assets (I/O ports, DMA channels, IRQ lines, memory-mapped I/O).

- Each driver must **request** the resources it intends to use.
- The kernel keeps a **global database** of allocated resources, allowing it to:
 - Prevent two drivers from claiming the same I/O address range.
 - Block autoprobe that would interfere with an already-configured device.
 - Resolve conflicts such as parallel-port vs. PLIP network driver.

If a reservation fails, the driver can either abort its initialisation (and be unloaded) or fall back to alternative resources.



Process Management in Linux

Process – the OS abstraction that holds all information needed to run a single program instance.

Linux adopts the traditional UNIX model (fork + exec) but adds a richer **threading** API via clone().

1. Process Identity

Identifier	Purpose
PID (process ID)	Unique numeric handle for the lifetime of the process.

PPID (parent PID)	Links a process to the one that created it.
Credentials	UID, GID and supplementary groups – define access rights.
Personality	Minor behavioural tweaks for compatibility with other UNIX variants.
Namespace	View of the filesystem hierarchy; processes can have distinct roots.
Process group / session IDs	Used by job control and signal distribution.

All identifiers are immutable except the **process group** and **session**, which may be changed after appropriate security checks.

2. Process Environment

- **Argument vector (argv)** – null-terminated list of command-line arguments; conventionally argv[0] holds the program name.
- **Environment vector (envp)** – list of NAME=VALUE strings (e.g., TERM=xterm, LANG=en_US.UTF-8).
- Both vectors reside at the top of the user stack; they are inherited across fork() and replaced when exec() supplies a new environment.

3. Process Context (dynamic state)

Sub-context	Contents
Scheduling context	CPU registers, kernel stack, priority, pending signals; saved/restored on a context switch.
Accounting	CPU time, memory usage, I/O statistics accumulated over the process's lifetime.
File descriptor table	Array of pointers to open struct file objects; indexed by integer file descriptors.
Filesystem context	Current working directory, root directory, and namespace.

Signal-handler table	Mapping of each signal to its disposition (ignore, default, or user handler).
Virtual-memory context	Page tables describing the process's private address space.

These sub-contexts are stored in separate kernel structures; the main `task_struct` holds pointers, enabling **shared sub-contexts** (used by threads).

4. Fork-Exec vs. Clone (Thread Model)

- `fork()` → creates a **new task** with **all sub-contexts copied** (no sharing).
- `execve()` → replaces the current address space with a new program image.
- `clone()` → flexible; caller specifies a bitmap of flags indicating which sub-contexts to **share** with the child:

Flag	Shared Sub-context
CLONE_VM	Memory space
CLONE_FS	Filesystem information (cwd, root)
CLONE_FILES	Open-file table
CLONE_SIGHAND	Signal handlers
CLONE_THREAD	Same thread group (makes the child a true thread)

When all four sharing flags are set, `clone()` behaves like a **thread** in other OSes; with none set it is equivalent to `fork()`.



Scheduling Fundamentals

Linux implements **preemptive, priority-based** scheduling with two distinct classes:

Class	Intended workload	Scheduling policy
Real-time (0-99)	Time-critical tasks	FIFO or Round-Robin (RR)
Time-sharing (-20 → 19)	General user processes	Completely Fair Scheduler (CFS)

1. Completely Fair Scheduler (CFS)

- No fixed *time slice*; each runnable task receives a **proportion of CPU time** based on its **weight** (derived from the nice value).
- **Target latency** (e.g., 10 ms) = interval in which every runnable task should run at least once.
- **Granularity** = minimum time a task may run, preventing excessive context-switch overhead when many tasks exist.

Weight calculation (simplified)

- Default nice0 → weight 1 (baseline).
- Lower nice (e.g., -5) → higher weight → larger CPU share.
- Higher nice (e.g., +5) → lower weight → smaller share.

The scheduler maintains a **red-black tree** keyed by virtual runtime; the leftmost node (the task that has consumed the least CPU relative to its weight) is selected next.

2. Real-time Scheduling

- **FIFO** – first-come, first-served; a higher-priority task runs until it blocks or voluntarily yields.
- **RR** – round-robin within the same priority; each task runs for a short quantum then moves to the end of its priority queue.

Linux's real-time class is **soft**: the kernel guarantees priority ordering but does **not** provide hard-deadline guarantees.



Kernel Synchronization & Concurrency

Linux kernels (≥ 2.6) are **fully preemptible**; therefore, kernel code must protect shared data against both **concurrent threads** and **interrupt handlers**.

1. Locking Primitives

Primitive	Typical Use	Remarks
Spinlock	Short critical sections on SMP systems	Must be held for brief periods; disables preemption on single-CPU.

Semaphore	Longer-duration locks, sleeping while waiting	Used when holding a lock could block for a noticeable time.
Preempt count (in thread_info)	Tracks nesting of disabling-preempt sections	Increment on lock acquisition, decrement on release; kernel preemptible only when count = 0.

Pattern for SMP spinlock

```
spin_lock(&lock);      // acquire, may spin
preempt_disable();     // prevent scheduler preemption
... critical section ...
preempt_enable();      // re-enable preemption
spin_unlock(&lock);    // release
```

2. Interrupt Handling: Top-Half / Bottom-Half

- **Top half** – the immediate ISR; runs with local interrupts disabled, must finish quickly.
- **Bottom half** – deferred work that can be scheduled after the ISR returns; may run with full interrupt enable.

Bottom-half mechanisms include **softirqs**, **tasklets**, and the newer **threaded interrupt handlers**. The kernel guarantees that a bottom half will never be pre-empted by another instance of the same bottom half, but it *can* be pre-empted by a top half of higher priority.

3. Disabling Preemption vs. Disabling Interrupts

- `preempt_disable()` / `preempt_enable()` – prevents the scheduler from switching tasks while retaining interrupt handling.
- `local_irq_disable()` / `local_irq_enable()` – masks the local CPU's interrupt lines; used when a critical section must not be interrupted at all (e.g., manipulating data also touched by ISRs).

Both techniques manipulate the **preempt count**; a non-zero count blocks kernel preemption.



Symmetric Multiprocessing (SMP) Evolution

- Early kernels used a **Big Kernel Lock (BKL)** – a single global spinlock protecting all kernel code, limiting concurrency.
 - As SMP systems grew, the BKL was **split** into many fine-grained locks (per-subsystem spinlocks, per-structure mutexes), dramatically improving scalability.
 - Modern kernels also employ **processor affinity** (preferring to run a task on the same CPU) and **load-balancing** (migrating tasks to idle CPUs) to keep each core busy without excessive migration overhead.
-

Memory Management Overview

Linux distinguishes **physical memory zones** to accommodate architectural constraints (e.g., DMA limits on older chips).

Zone	Typical Constraints	Typical Use
ZONE_DMA	Low-addressable memory (< 16 MiB on 32-bit x86) needed for legacy DMA devices.	DMA buffers for ISA-compatible hardware.
ZONE_DMA32	Addresses below 4 GiB on 64-bit systems that still require 32-bit DMA.	Modern PCI devices on 64-bit machines.
ZONE_NORMAL	Direct-mapped memory reachable by the kernel without special handling.	Most kernel allocations and low-memory user pages.
ZONE_HIGHMEM	Memory not permanently mapped into kernel address space (only on 32-bit systems with > 4 GiB RAM).	High-capacity RAM used via temporary page-table mappings.

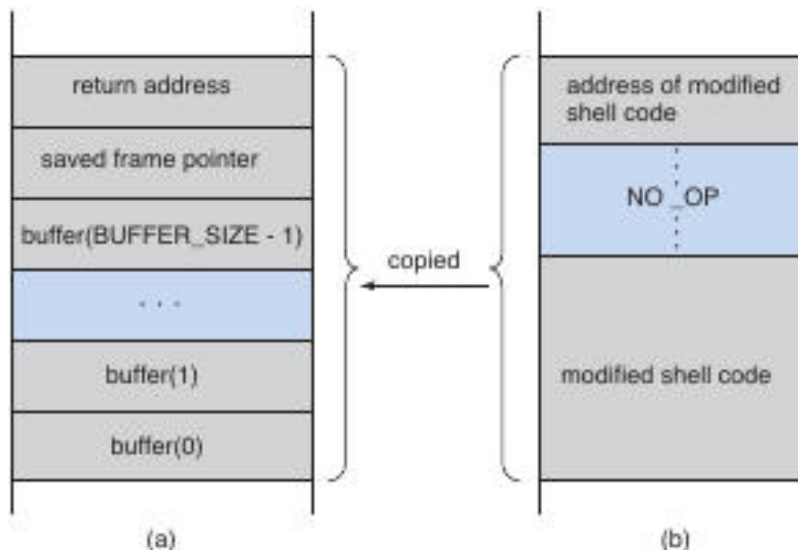
The **buddy allocator** operates within each zone, handling page-size allocations and ensuring that physical pages are kept in the appropriate zone based on the request's constraints (e.g., GFP_DMA flag forces allocation from ZONE_DMA).

Memory Zones & Physical Allocation

Zone – a class of physical memory defined by architectural constraints that determines which allocator may service a request.

- **ZONE_DMA** – low-address memory (typically the first 16 MiB) accessible by legacy DMA-capable devices.
- **ZONE_DMA32** – the first 4 GiB of physical memory on systems that can address only 32 bits for DMA.
- **ZONE_HIGHMEM** – portions of RAM that are *not* permanently mapped into the kernel's linear address space; used on 32-bit systems where the kernel can map only a subset (e.g., the first 896 MiB) of total RAM.
- **ZONE_NORMAL** – the “regular” region that is directly mapped for kernel use; on modern x86-64 systems it comprises essentially all RAM because the kernel address space is large enough to hold it all.

The relationship of zones to physical addresses is illustrated in the diagram below (Figure 15.3).



The figure shows how a 32-bit system partitions its RAM into the four zones, with the kernel mapping the lower zones directly and treating the remainder as high memory.

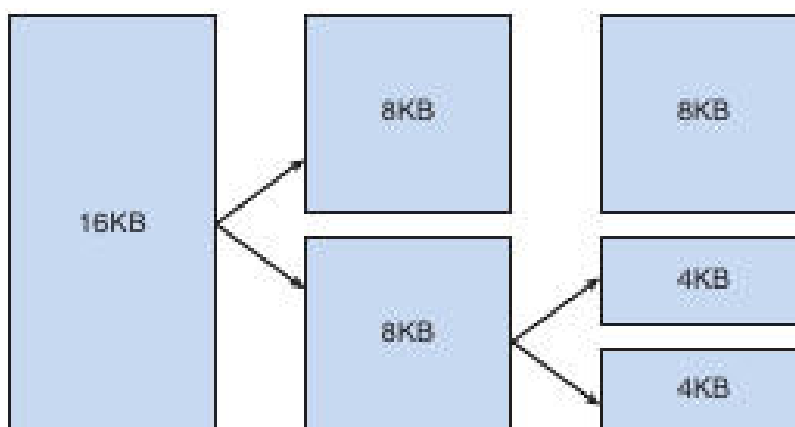
When a request for physical memory arrives, the kernel consults the **zone lists** and selects the appropriate allocator for that zone. Each zone has its own **buddy allocator** (see next section) that can also satisfy requests for physically contiguous pages when needed.

Buddy Allocator

Buddy system – a binary-tree-like allocator that keeps free memory in power-of-two sized blocks and merges adjacent “buddies” when both are free.

- The smallest allocatable unit is a **page** (4 KiB).
- Free blocks are kept in **linked lists**, one list per allowed size (4 KiB, 8 KiB, 16 KiB, ...).
- **Allocation:**
 1. Find the smallest list that can satisfy the request.
 2. If the list is empty, recursively split a larger block (its *buddy*) until a block of the desired size appears.
- **Freeing:**
 1. Return the block to its size-class list.
 2. Check whether its buddy is also free; if so, merge them into the next-larger size class.

The following diagram visualizes a typical buddy-heap split from a 16 KiB block down to the 4 KiB pieces needed for a request.



The tree-like split shows how larger blocks are divided into halves (“buddies”) until a suitably sized free block is available.

All kernel memory allocations ultimately flow through this buddy system, either directly (for page-level requests) or indirectly via higher-level allocators.

Kernel Memory Allocators

kmalloc / kfree

kmalloc – a kernel analogue of malloc that obtains one or more physical pages from the buddy allocator and then parcels them into smaller objects of the requested size.

- The allocator maintains **size-specific free lists** (slab caches).
- On a request, the appropriate list is consulted; if empty, a fresh page is taken from the buddy allocator and split.
- Memory obtained with kmalloc remains allocated until the caller explicitly releases it with **kfree**; the allocator does not automatically reclaim unused fragments.

Slab Allocator

Slab – a cache-oriented allocator optimized for objects of a fixed size (e.g., inode structures, task structs).

Concept	Role
Cache	Holds pre-allocated objects of a particular size; each cache serves one kernel data-type.
Slab	One or more <i>physically contiguous</i> pages that back a cache. A slab can be full , partial , or empty .
Object	An individual kernel structure taken from a cache (e.g., a struct task_struct).

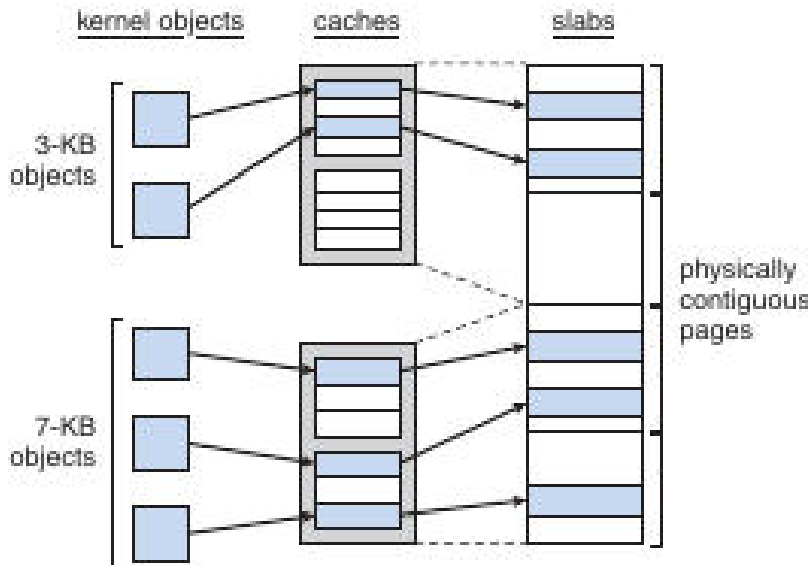
The slab lifecycle:

1. **Cache creation** – the kernel defines a cache for a specific object size.
2. **Slab allocation** – one or more contiguous pages are obtained from the buddy allocator to back the cache.
3. **Object distribution** – objects are carved out of the slab; free objects are kept on a per-cache freelist.

4. **State transitions** – a slab moves between *empty* → *partial* → *full* as objects are allocated and freed.

When a cache runs out of free objects, a new slab is allocated; when a slab becomes completely empty, it can be returned to the buddy system.

The following diagram depicts the relationship among kernel objects, caches, and slabs.



Three 3-KB objects and three 7-KB objects are allocated from their respective caches, which in turn draw memory from slabs made of contiguous 4-KB pages.

Page Cache & Virtual Memory

Page Cache

- The **page cache** is the kernel's primary cache for block-device I/O.
- Entire pages of file contents are cached; when a process reads a file, the kernel first looks for the page in the cache and only issues a disk read if the page is absent.
- The cache is shared across all processes; any modification to a cached page is instantly visible to every process that maps the same file region.

Virtual Memory Regions

vm_area_struct – a kernel data structure describing a contiguous, page-aligned region of a process’s virtual address space.

Key attributes stored in a vm_area_struct:

- Start and end virtual addresses.
- Access permissions (read/write/execute).
- Backing store type (file-backed, anonymous).
- Pointers to a **operations table** that implements page-fault handling for the region.

Two complementary views of a process’s address space exist:

1. **Logical view** – a balanced binary tree of vm_area_structs, enabling fast lookup of the region that contains a given virtual address.
2. **Physical view** – the hardware page tables that map virtual pages to either physical RAM or swap space.

Backing Store Types

Backing type	Description
Anonymous (no backing)	Pages are zero-filled on first access; used for heap, stack, and brk expansions.
File-backed	Pages correspond to data in the kernel’s page cache; reading/writing a page updates the underlying file.
Shared vs. Private	<i>Shared</i> mappings propagate writes to all processes mapping the same region; <i>private</i> mappings trigger copy-on-write (COW) when a write occurs, preserving isolation.

When a COW fault occurs, the kernel allocates a new physical page, copies the original contents, updates the page table for the faulting process, and marks the new page as writable.

Process Address Space Creation

exec() – loading a new program

- The exec() system call replaces the current process image with a new executable.
- The kernel checks execute permission on the target file, selects an appropriate **binary loader** (e.g., ELF loader), and sets up the initial virtual-memory mappings (text, data, BSS, stack).
- No physical pages are loaded immediately; pages are brought in on demand via **demand paging** (page-fault handling).

fork() – cloning an existing process

- fork() creates a child process that initially shares the parent's address space.
- The kernel copies the parent's vm_area_struct descriptors and creates a separate set of page-table entries for the child.
- All shared pages are marked **read-only** and flagged for **copy-on-write**.
- When either process writes to a COW page, the kernel allocates a new physical page for the writer, preserving the other process's view.

This strategy maximizes sharing of memory (code, read-only data) while preserving isolation for writable data.



Swapping & Paging

- Linux employs **paging** (individual page movement) rather than whole-process swapping.
- The page-out algorithm is a **modified second-chance (clock)** scheme: each page has an **age** counter that reflects recent activity.
 - Frequently accessed pages acquire higher age values; pages with low age are candidates for eviction.
- When a page is selected for eviction, the kernel writes its contents to a **swap device** (or swap file) using a bitmap to track used swap blocks.
- The page-table entry's **present-bit** is cleared and the entry is repurposed to store the swap-slot index, enabling lazy re-allocation on the next fault.

Swap to a dedicated partition is faster than swap to a regular file because the latter incurs extra filesystem overhead.



Kernel Virtual Memory

- The kernel reserves a contiguous region of its own virtual address space for internal use.
 - The **low-mem map** contains a one-to-one mapping of every physical page, allowing simple physical-to-virtual translations.
 - The **kernel code and data** allocated by the normal page allocator reside here.
- The remaining kernel virtual space is available for **vmalloc**-style allocations:
 - `vmalloc()` returns virtually contiguous memory that may be physically scattered; useful for large buffers that do not require DMA.
 - `vmap()` maps device I/O memory (memory-mapped I/O) into the kernel's address space.

All kernel pages are protected by the MMU; page-table entries for kernel pages are marked as **kernel-only**, preventing user-mode access.



Execution & Loading of User Programs

- Linux maintains a **loader table** of functions, each capable of handling a particular executable format.
 - Historically this supported **a.out**; modern systems primarily use the **ELF** format.
 - The loader parses the ELF header, creates separate virtual-memory regions for each *segment* (text, data, rodata, BSS, etc.), and registers the entry point as the initial program counter.
 - Because the loader only sets up mappings, the actual page data are fetched on first access via page faults (demand paging).
-



ELF Program Layout

Region	Typical contents	Permissions
Stack (high addresses, grows down)	Argument list, environment strings, return addresses	Read/Write
Text	Executable code	Read/Execute (write-protected)
RO-data	Constant literals, read-only globals	Read/Execute
Data	Initialized writable globals	Read/Write

BSS	Uninitialized globals (zero-filled)	Read/Write (demand-zero)
Heap (brk region)	Dynamically allocated memory via sbrk/brk	Read/Write

The loader also creates a **guard page** to catch stack overflows and establishes the initial **program-counter** from the ELF entry point.

Static vs. Dynamic Linking

- **Static linking** embeds all required library code into the executable at build time; the program can start immediately but each binary carries its own copy of common library code, wasting memory and disk space.
 - **Dynamic linking** keeps libraries in separate **shared objects** (.so files).
 - The executable contains a small **runtime linker stub** that, on start-up, loads the necessary shared objects, resolves symbols, and maps them into the process’s address space.
 - Shared libraries are compiled as **position-independent code (PIC)**, allowing them to be placed at any virtual address without relocation.
 - Because the same physical pages can be mapped into multiple processes, dynamic linking reduces overall memory pressure.
-

File Systems & the Virtual File System (VFS)

VFS – an abstraction layer that presents a uniform API to the kernel while delegating actual file-system operations to specific implementations (e.g., ext3, NFS).

Core VFS Objects

Object	Purpose	Typical fields
inode	Represents a single file or directory; stores metadata (owner, size, timestamps) and pointers to data blocks.	i_mode, i_uid, i_size, i_blocks, i_ops

file	Represents an open file descriptor; tracks current file position and access flags.	f_pos, f_flags, f_ops
dentry	Represents a directory entry (the name-to-inode mapping).	d_name, d_inode, d_parent, d_ops
superblock	Represents a mounted file-system instance; holds global info and provides functions to locate inodes.	s_dev, s_root, s_ops

Each object type defines a **function table** (e.g., inode_operations, file_operations). The VFS performs an operation by invoking the appropriate function from the object's table, without needing to know the underlying file-system type.

- **File operations** include open(), read(), write(), mmap(), etc.
- **Inode operations** cover actions such as lookup(), create(), link(), and unlink().

Interaction Flow

1. A system call (e.g., read(fd, ...)) arrives.
2. The kernel retrieves the file object associated with fd.
3. It calls file->f_op->read(...).
4. The read implementation may invoke inode->i_op->readpage to fetch data from the underlying storage.

Because the VFS caches inode and dentry objects, repeated lookups are fast, and the kernel can maintain consistency across different file-system types.

Dentry Objects & Pathname Resolution

Dentry – a kernel object that represents a single component of a pathname (e.g., “/”, “usr”, “include”, “stdio.h”).

- Each component of a path is cached in its own dentry structure.
- When a process opens **/usr/include/stdio.h**, the VFS must obtain the inode for “/”, then look up the dentry for “usr”, continue to “include”, and finally “stdio.h”.

- Because pathname translation can be expensive, Linux keeps a **dentry cache**. Retrieving an inode from this cache is far faster than reading the directory from disk each time.

ext3 (and ext4) File System

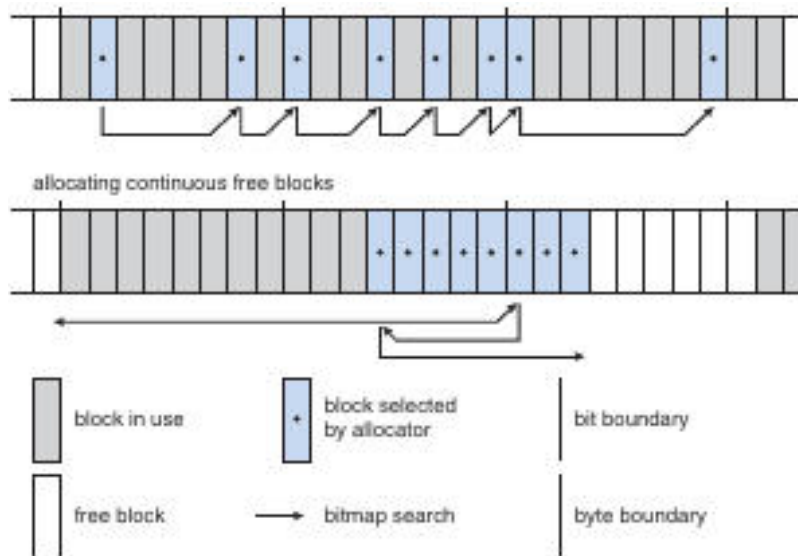
ext3 – the third extended file system, the historically dominant on-disk format for Linux. It evolved from the Minix FS → ext2 → ext3 (added journaling) → ext4 (adds extents, larger volumes, etc.).

Directory & Inode Layout

- Directories are ordinary files; each block contains a series of **directory entries**:
 - *entry length*
 - *inode number* (points to the inode of the named file)
 - *filename*
- Inodes store metadata and an array of **direct, indirect, double-indirect, and triple-indirect block pointers** (up to three levels of indirection).
- Unlike BSD Fast File System (FFS), ext3 **does not use fragments**; every block is a full allocation unit (1 KB – 8 KB).

Block Allocation Policies

- Ext3 groups blocks into **block groups** (conceptually similar to cylinder groups).
- Allocation steps:
 1. **Select block group** – for data blocks, the group containing the file's inode is preferred; for new inodes, the parent directory's group is used.
 2. **Search bitmap** – locate a free byte in the group's bitmap. If none, move to the next group.
 3. **Backward extension** – once a free byte is found, scan backward until a previously allocated block is encountered; this avoids creating holes.
 4. **Pre-allocate up to eight contiguous blocks** – the allocator extends the allocation forward (up to 8 blocks) to improve sequential I/O and reduce fragmentation.
- The bitmap-search process is illustrated below:



The diagram demonstrates how the allocator finds a free byte, backs up to the previous allocation, and then pre-allocates a block run to keep data together.

- **Fragmentation mitigation** – by keeping related blocks in the same group and pre-allocating runs, ext3 reduces seek time and improves read-ahead performance, even though some fragmentation inevitably occurs.

Journaling

Journaling – a technique where modifications are first written to a sequential log (the journal) before being applied to the main file-system structures.

- A **transaction** groups a set of operations.
- Steps:
 1. Write transaction data to the journal.
 2. Mark the transaction **committed** once the journal entry is on disk.
 3. Replay the transaction to update the actual file-system metadata.
- The journal is a circular buffer; it may reside on the same device, a separate partition, or even a dedicated disk spindle to reduce head contention.
- On a crash, any **uncommitted** transactions are discarded, and **committed but unapplied** transactions are replayed, guaranteeing a consistent file system.

- Because writes go first to the sequential journal, metadata-heavy operations (create, delete, rename) are faster than on non-journaled file systems.

/proc Virtual File System

/proc – a pseudo-file system that presents kernel and process information as regular files.

- Each running process appears as a directory named by its PID (e.g., **/proc/1234**).
- Inodes for these entries are generated dynamically; the upper 16 bits of the 32-bit inode encode the PID, the lower bits identify the specific file (e.g., **/proc/1234/status**).
- Global (non-process-specific) entries (kernel version, memory stats, etc.) use a PID field of zero.
- The VFS treats these files like any other, allowing tools such as `ps` to read process state without privileged access.
- A special subtree **/proc/sys** holds tunable kernel variables; they can be read/written as plain text or accessed efficiently via the `sysctl()` system call, which operates on the same underlying dynamic entry tree.

Device I/O Model

Linux abstracts all hardware devices as files, enabling uniform open/read/write semantics.

Block Devices

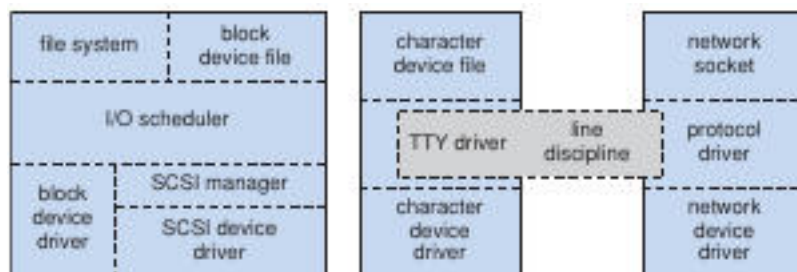
- Provide **random access** to fixed-size blocks (hard disks, SSDs, CD/DVD, flash).
- Access is mediated by the **I/O scheduler**, which orders requests to minimize seek time.

I/O Scheduling Algorithms

Algorithm	Characteristics	Typical Use
Elevator (C-SCAN)	Requests sorted by start-sector; head moves in	Simple workloads, legacy systems.

	one direction then jumps back.	
CFQ (Completely Fair Queueing)	Maintains a separate request queue per process; services queues round-robin, each getting a configurable quantum.	Interactive workloads where per-process fairness and low latency are important.

- The scheduler merges adjacent requests to form larger I/O operations, reducing per-request overhead.



The diagram places the block-device subsystem within the larger kernel structure, highlighting the scheduler's role between the VFS and the physical driver.

Character Devices & TTY

- Character devices deliver a **stream of bytes** (e.g., keyboards, serial ports).
- Drivers register a set of file-operation callbacks (read, write, open, close).
- **TTY subsystem** adds line-discipline processing (canonical mode, echo, flow control).
- Terminal line disciplines can also implement network protocols (PPP, SLIP), appearing to the networking stack as network-device drivers while still being accessed through the character-device interface.

Network Devices

- Network interfaces are not accessed directly as files; instead, applications use the **socket interface**.
- The kernel's network stack consists of:
 1. **Socket layer** – BSD-compatible API (socket(), bind(), connect(), etc.).

2. **Protocol drivers** – TCP, UDP, IP, etc., handling packet processing, routing, and error handling.
 3. **Network-device drivers** – hardware-specific code that moves packets to/from the NIC.
- Data moves between layers encapsulated in a **skbuff** (socket buffer) structure, which contains pointers into a contiguous buffer allowing header manipulation without copying.

Interprocess Communication (IPC)

Signals & Synchronization

Signal – an asynchronous notification sent to a process to indicate an event (e.g., SIGCHLD when a child exits).

- Signals can be sent by any process to another process that shares the same UID or by the kernel (e.g., timer expiry, I/O completion).
- Only the occurrence of the signal is conveyed; detailed information must be obtained via other mechanisms.
- Kernel-mode asynchronous events use **wait queues** and **process states** rather than signals.

Pipes & Semaphores

- **Pipe** – a unidirectional byte stream represented by a pair of inodes; the read and write ends each have their own wait queue for synchronization.
- **System V semaphores** – integer counters that can be atomically incremented/decremented; multiple semaphores can be grouped into a set, allowing complex synchronization patterns.
- The kernel's wait-queue implementation underlies both pipes and semaphore operations, ensuring that blocked processes are awakened when the corresponding condition is satisfied.

Shared Memory

- **Shared memory segment** – a persistent kernel object that multiple processes can map into their address spaces.
- Once mapped, reads/writes are instantaneous; however, the kernel provides **no inherent ordering or mutual exclusion**.

- Typically combined with semaphores or mutexes for safe concurrent access.
- Shared-memory objects are pageable: the kernel may swap them out like any other page, and they persist even when no process currently maps them.

Linux Networking Stack Details

- All network traffic is carried in **skbuff** structures.
 - The skbuff holds pointers to the packet's data and metadata (protocol headers, length, checksum).
 - Because the data can start/end anywhere within the buffer, the stack can prepend or strip headers without copying.
- The **socket layer** presents a uniform API; applications do not need to know whether the underlying protocol is TCP, UDP, or a custom one.
- **Protocol drivers** may rewrite, fragment, or discard packets before passing them upward (to sockets) or downward (to device drivers).
- This design mirrors the earlier **VFS abstraction**: the networking stack hides hardware details behind a consistent interface.

Comparative Tables

File-System Features (ext3 vs. ext4)

Feature	ext3	ext4
Journaling	Yes (metadata-only or full)	Yes (improved journal handling)
Extent support	No	Yes – reduces fragmentation
Maximum file size	2 TB	16 TB
Block sizes	1 KB – 8 KB	Up to 64 KB
Allocation groups	Fixed-size block groups	Larger “flexible” block groups for better scalability

Device Types & Access Semantics

Device class	Access pattern	Example devices
--------------	----------------	-----------------

Block	Random access to fixed-size blocks	HDD, SSD, CD-ROM
Character	Serial stream of bytes	Keyboard, mouse, serial port
Network	Packet-oriented via sockets	Ethernet NIC, Wi-Fi adapter

These notes extend the earlier discussion of the VFS, access-matrix concepts, and capability-based protection by showing how Linux implements concrete objects (inodes, dentries, block groups) and runtime mechanisms (journaling, I/O scheduling, IPC) to enforce security policies and maintain performance.

IP Layer & Routing

IP (Internet Protocol) – provides host-to-host routing across heterogeneous networks. It sits beneath the transport protocols UDP, TCP and ICMP.

UDP, TCP, ICMP positioning

- **UDP** – connectionless, carries independent datagrams.
- **TCP** – establishes reliable, in-order streams; handles automatic retransmission.
- **ICMP** – conveys error and status messages (e.g., “destination unreachable”).

Packet identification in the networking stack

- Each incoming packet is wrapped in an **skbuff** structure that already carries an internal protocol identifier.
- Device drivers decode the hardware-specific protocol field (e.g., Ethernet type) and look up the corresponding kernel protocol handler.
- New protocols can be introduced as **loadable kernel modules** that register their identifier.

Routing responsibilities of the IP driver

1. **Lookup** the destination in the **Forwarding Information Base (FIB)** – a set of hash tables indexed by destination address.
2. If the FIB entry is missing, consult the **route-cache** (a smaller hash storing recent successful lookups).

3. Upon a cache miss, perform a full routing computation and insert the result into the cache (entries expire after a period of inactivity).

Structure	Scope	Typical contents
FIB	Persistent	Static routes, default gateway, network masks
Route cache	Transient	Recent destination → next-hop mappings; expires after idle time

Firewall integration

- After routing decisions, the IP layer may invoke the **firewall subsystem**.
- The firewall maintains several **chains** (ordered rule lists):
 - **INPUT** – packets destined for the local host.
 - **OUTPUT** – packets generated locally.
 - **FORWARD** – packets being routed through the host.
- Each rule specifies a **decision** (accept, drop, reject) and matching criteria (source/destination address, protocol, ports, etc.).

Chain	Purpose
INPUT	Filter inbound traffic to the host
OUTPUT	Filter locally generated traffic
FORWARD	Filter traffic that traverses the host

Fragmentation & reassembly

- **Outgoing**: If a packet exceeds the MTU of the egress device, the IP driver splits it into fragments, each with its own header.
- **Incoming**: The driver maintains an **ipfrag** object per incomplete datagram and an **ipq** (fragment queue) keyed by source/destination address, ID, and protocol.
 - Matching fragment → added to the appropriate **ipq**.
 - When the final fragment arrives, a new **skbuff** is assembled and re-injected into the IP processing path.



Transport Layer: UDP & TCP Socket Handling

Sockets – kernel objects that bind a transport protocol to a pair of IP addresses and ports.

Addressing sockets

- A socket is uniquely identified by four values: **source IP**, **source port**, **destination IP**, **destination port**.
- The kernel stores socket descriptors in **hash tables** keyed on this 4-tuple for fast lookup of incoming packets.

UDP handling

- Stateless: each datagram is delivered to the matching socket (if any) and discarded otherwise.

TCP reliability mechanisms

- Maintains two ordered lists per connection:
 1. **Unacknowledged outbound segments** – retransmitted after a timeout.
 2. **Out-of-order inbound segments** – buffered until missing data arrive, then delivered in order to the socket.

These mechanisms build on the earlier **access-matrix** concept: the kernel enforces per-socket rights (read/write) before delivering data.

Linux Security Model Overview

Authentication

Classic password file

Passwords are stored as a salted one-way hash.

- The file contains : entries.
- On login, the presented password is combined with the stored salt, hashed, and compared to the stored value.
- Weaknesses of early UNIX implementations: 8-character limit, limited salt space → susceptible to dictionary attacks.

Enhancements

- **Extended password files** – not world-readable, support longer passwords, stronger hash algorithms.
- **PAM (Pluggable Authentication Modules)** – a shared-library framework allowing system components to load authentication, account, session, and password-changing modules via a central configuration file. Adding a new mechanism only requires updating the PAM config; all services instantly gain the capability.

2 Access Control

UID/GID model

- Every object (file, device, IPC primitive) carries a **UID** (owner) and a **GID** (group).
- A process has:
 - **Real UID/GID** – identity of the user who launched it.
 - **Effective UID/GID** – identity used for permission checks (may differ due to `setuid/setgid`).
 - **Saved UID/GID** – can be restored after a temporary privilege change.
 - **fsuid/fsgid** – used exclusively for filesystem permission checks, allowing a process to act on behalf of another user without changing its real/effective IDs.

Identifier	Role
Real UID/GID	Original user identity
Effective UID/GID	Determines current access rights
Saved UID/GID	Restorable after privilege drop
fsuid/fsgid	Filesystem-specific rights; independent of real/effective IDs

- **Permission bits** (read, write, execute) are evaluated in the order: owner → group → others.
- The **root UID (0)** bypasses normal checks and can access any object.

Setuid/setgid behavior

- Executable's **setuid** bit causes the **effective UID** of a process to become the file's owner on exec.
- Linux extends the classic model:
 - Supports the **saved UID** mechanism for repeated dropping/re-acquiring of privileges.

- Introduces **fsuid/fsgid** to limit filesystem access without granting full super-user rights (useful for daemons serving files on behalf of clients).

File-Descriptor Passing

A process can transfer an open file descriptor to another process via a local Unix domain socket.

- The receiving process obtains a duplicate descriptor that refers to the same underlying kernel object, inheriting the same access rights.
- This mechanism enables fine-grained delegation (e.g., a print client passes only the files to be printed to the printer daemon, without giving the daemon broader filesystem access).



Integrated View of Protection Mechanisms

- The **access matrix** introduced earlier maps *domains* (processes/UIDs) to *objects* (files, sockets) with specific rights; the modern Linux implementation realizes this matrix through UID/GID ownership, permission bits, and capability-like features (fsuid/fsgid, setuid).
- **Capability lists** are mirrored by the kernel's ability to load protocol modules and firewall rules dynamically, each module holding the rights needed to operate on particular network objects.
- **RBAC** concepts appear in PAM, where modules can enforce role-based policies before authentication succeeds.

These layers—routing, transport, and security—work together to enforce **least privilege**, **defense in depth**, and **dynamic extensibility** across the Linux networking stack.



File Systems & Disk Storage

File system – an organized collection of data structures that manage how files are stored, retrieved, and organized on a storage medium.

Block Allocation Strategies

Strategy	Description	Typical Use
----------	-------------	-------------

Direct blocks	Each inode contains pointers that directly reference data blocks.	Small files; fast access.
Single-indirect block	A pointer in the inode references a block that holds an array of data-block pointers.	Extends file size beyond direct block limit.
Double-indirect block	Two levels of indirection; inode → indirect block → second-level block → data blocks.	Very large files.
Indexed allocation	A dedicated index block contains an ordered list of data-block numbers.	Improves sequential read/write performance.
Linked allocation	Each data block stores the address of the next block in the file.	Simple implementation; suffers from poor random access.
Contiguous allocation	File occupies a set of consecutive blocks on disk.	Maximizes throughput; suffers from external fragmentation.

File-system Structures

- **Inode** – stores metadata (owner, permissions, timestamps) and block pointers.
- **Directory** – often represented as an **acyclic graph** (no cycles) linking names to inodes; can be generalized to a **graph** where hard links create multiple paths.
- **File-allocation table (FAT)** – a simple linked-list of block numbers; each entry points to the next block of a file.
- **Ext2/Ext3/Ext4** – use block groups, inode tables, and (for Ext3/4) a **journal** to ensure consistency after crashes.
- **Journaling** – writes intent (metadata changes) to a sequential log before applying them to the main file system; on recovery, committed entries are replayed.

Disk-Related Metadata

- **Boot block** – first sector containing bootstrap code; essential for system start-up.
- **Bad-block table** – records defective sectors to avoid using them.

- **Master Boot Record (MBR)** – contains partition table and boot loader; located in the first sector of a disk.
- **Master File Directory (MFD)** – top-level directory entry in some file systems.

Disk Scheduling Algorithms

Algorithm	Principle	Typical Performance Impact
FCFS (First-Come-First-Served)	Service requests in arrival order.	Simple; can cause long average seek times.
SSTF (Shortest-Seek-Time-First)	Choose request with minimal head movement.	Reduces average seek, may cause starvation.
SCAN (Elevator)	Move the head in one direction, servicing all requests, then reverse.	Balances latency; reduces starvation.
C-SCAN	Like SCAN but after reaching end, head returns quickly without servicing.	Provides more uniform wait times.
LOOK / C-LOOK	Same as SCAN/C-SCAN but reverse direction at the last request instead of disk edge.	Further reduces unnecessary travel.
EDF (Earliest-Deadline-First)	Prioritize requests with earliest deadline (real-time).	Guarantees deadline compliance when feasible.
LOTTERY	Assign tickets to requests; random draw determines next service.	Provides probabilistic fairness.

RAID & Network Storage

- **RAID levels** – combine multiple disks for redundancy or performance (e.g., RAID 0 striping, RAID 1 mirroring, RAID 5 parity).
 - **SAN (Storage-Area Network)** – block-level storage accessed over a dedicated high-speed network (Fibre Channel, iSCSI).
 - **NAS (Network-Attached Storage)** – file-level sharing using protocols such as NFS or SMB.
-

Memory Management & Direct Access

Direct Memory Access (DMA) – a hardware feature that allows peripheral devices to read/write main memory without CPU intervention.

DMA-Related Concepts

- **DMA controller** – orchestrates address and data transfer between device and memory.
- **Direct Virtual Memory Access (DVMA)** – extends DMA to virtual memory by translating virtual addresses for the device.
- **Distributed Lock Manager (DLM)** – coordinates lock acquisition across nodes in a distributed system; can be built on top of DMA for high-speed data movement.

Physical Memory Types

- **DRAM (Dynamic RAM)** – standard main memory, requires periodic refresh.
- **NVRAM (Non-volatile RAM)** – retains data without power; used for fast persistent storage.
- **Cache hierarchy** – typically L1/L2/L3 caches; employ **write-back** policies tracked by **dirty bits** (bits indicating modified cache lines).

Virtual Memory & Page Management

- **Page table** – maps virtual pages to physical frames; can be **hierarchical** (multi-level) or **inverted** (single table indexed by frame).
- **Buddy allocator** – splits memory into power-of-two blocks; merges *buddies* when both are free, reducing fragmentation.
- **Slab allocator** – caches objects of the same size; each *slab* consists of one or more pages.
- **kmalloc / kfree** – kernel analogues of malloc/free; allocate from the slab system.

Page-Replacement Policies

Policy	Key Idea	Typical Use
FIFO	Evict the oldest loaded page.	Simple hardware implementation.

LRU (Least-Recently-Used)	Evict page not used for longest time.	Good approximation of optimal policy.
LFU (Least-Frequently-Used)	Evict page with fewest accesses.	Works for workloads with stable hot spots.
Optimal	Evict page whose next use is farthest in the future (theoretical).	Benchmark for evaluating other policies.
Clock (second-chance)	Circular list; give a second chance if referenced bit set.	Low overhead, approximates LRU.

- **Effective Access Time (EAT)** – weighted average of memory-access time and page-fault service time:

$$EAT = (1-p) \times t_{\text{mem}} + p \times t_{\text{fault}}$$
where p is the page-fault probability.

I/O Buffering & Double Buffering

- **Double buffering** – while one buffer is being filled by the device, the other is processed by the CPU; reduces idle time.



Scheduling & Real-Time Concepts

Scheduling – the OS decision process that determines which ready tasks receive CPU time (or disk time) and when.

Disk-Scheduling Recap (see 📁 section)

CPU Scheduling Algorithms

Algorithm	Category	Main Feature
FCFS	Non-preemptive	Simple queue; can cause high average turnaround.
RR (Round-Robin)	Preemptive	Fixed time quantum; fair sharing.

SJF (Shortest Job First)	Non-preemptive	Minimizes average waiting time; requires job length knowledge.
Priority scheduling	Preemptive / Non-preemptive	Tasks assigned static or dynamic priorities.
Multilevel Queue	Hybrid	Separate queues for different priority classes (e.g., interactive vs batch).
Feedback Queue	Adaptive	Tasks that use up their quantum are demoted to lower-priority queues.
EDF (Earliest-Deadline-First)	Real-time	Guarantees deadline satisfaction if total utilization ≤ 1 .
Rate-Monotonic	Real-time	Fixed-priority based on task period; optimal for static-priority real-time systems.
Lottery	Probabilistic	Tickets assigned to processes; random draw yields execution.

- **Context-switch latency** – time overhead incurred when the scheduler saves the state of the current task and loads the next.

Security, Encryption & Protection

Security – mechanisms that preserve confidentiality, integrity, and availability of system resources.

Cryptography Basics

Primitive	Goal	Typical Algorithm
Symmetric encryption	Confidentiality with shared secret	AES, 3DES

Asymmetric encryption	Confidentiality & key exchange without pre-shared secret	RSA, ECC
Hash functions	Integrity verification, password storage	SHA-256, SHA-3
Message-Authentication Code (MAC)	Integrity + authentication using secret key	HMAC-SHA256
Digital signatures	Non-repudiation, authenticity	RSA-DS, ECDSA

- **ECC (Error-Correcting Code)** – adds redundancy to detect/correct bit errors (e.g., Reed-Solomon in RAID).

Password Security

- **Hashing** – store only hash(salt || password).
- **Salting** – random per-user value concatenated before hashing to thwart pre-computed dictionary attacks.
- **One-Time Passwords (OTP)** – a value valid for a single authentication event (challenge-response).

Access Control & RBAC

- **RBAC (Role-Based Access Control)** – permissions are assigned to *roles*; users acquire roles to gain privileges.
- **Effective UID** – the UID the kernel uses for permission checks (may differ from real UID due to setuid).

Legal & Policy Mechanisms

- **DMCA** – U.S. Digital Millennium Copyright Act; defines anti-circumvention provisions.
- **DRM (Digital Rights Management)** – technologies that enforce usage restrictions on digital content.
- **DMZ (Demilitarized Zone)** – network segment separating internal LAN from external networks, typically hosting public services.

Threats & Countermeasures

Threat	Typical Vector	Mitigation
--------	----------------	------------

Virus	Self-replicating code attached to executables or macros.	Antivirus signatures, heuristic detection, code signing.
Worm	Network-propagating exploits (e.g., buffer overflow, remote shell).	Patch management, intrusion-detection systems (IDS).
DoS / DDoS	Flooding resources, SYN floods, application-layer abuse.	Rate limiting, firewalls, traffic scrubbing services.
Man-in-the-Middle	Intercepting and altering traffic.	TLS encryption, certificate validation.
Privilege escalation	Exploiting bugs (e.g., buffer overflow) to gain higher rights.	Address Space Layout Randomization (ASLR), non-executable stack, SELinux/AppArmor.

Linking, Loading & Dynamic Libraries

Dynamic linking – the process of resolving external symbols at run-time rather than at compile-time.

Library Types

Type	File Extension	Loading Time
Static library	.a (Unix) / .lib (Windows)	Linked into executable; no run-time dependency.
Dynamic Link Library (DLL)	.so (Unix) / .dll (Windows)	Loaded by the dynamic linker when the program starts or on demand.
Dynamic loading	dlopen / LoadLibrary APIs	Modules can be loaded/unloaded during execution.

- **Dynamic Symbol Resolution** – the loader searches the *global symbol table* for needed functions; unresolved symbols cause load failure.

- **Position-Independent Code (PIC)** – compiled so that it can execute at any address, facilitating shared-library loading.

Drivers, I/O Subsystems & Interrupts

Device driver – kernel module that abstracts hardware specifics behind a uniform interface.

Device Categories

Category	Access Model	Example
Block device	Random-access reads/writes of fixed-size blocks.	Hard disk, SSD, RAID controller.
Character device	Stream of bytes; may be unbuffered.	Keyboard, serial port, /dev/null.
Network device	Packets transmitted/received via socket API.	Ethernet NIC, Wi-Fi adapter.

I/O Scheduling (Block Layer)

- **CFQ (Completely Fair Queuing)** – assigns each process a time slice of the disk, achieving per-process fairness.
- **Elevator (C-SCAN)** – groups requests to reduce seek latency; used by many traditional kernels.

Interrupt Handling

- **Top-half ISR** – quick handling; disables further interrupts on the local CPU.
- **Bottom-half** – deferred work (softirqs, tasklets, threaded IRQs) executed later with full interrupt enable.
- **Interrupt priority levels** – maskable vs. non-maskable (NMI) interrupts; higher levels pre-empt lower ones.

Driver Registration (Linux)

- Drivers register with the kernel via `register_chrdev`, `pci_register_driver`, etc.

- Unregistration removes entries from the kernel's internal tables, preventing further use.
-

Miscellaneous OS Concepts

Kernel Modules & Loadable Code

- **Loadable Kernel Module (LKM)** – compiled object inserted at run-time (insmod, rmmod).
- **Conflict resolution** – the module manager ensures no two modules claim the same hardware resources.

Process Model (Linux)

- **fork()** – creates a child with copy-on-write semantics; shares address space until a write occurs.
- **exec()** – replaces the current address space with a new program image; permissions are checked before loading.
- **Process identifiers** – PID, PPID, real/effective/saved UID/GID, and fsuid/fsgid for filesystem checks.

Synchronization Primitives

- **Spinlock** – busy-wait lock for short critical sections on SMP systems.
- **Semaphore** – can block the calling thread; used for longer waits.
- **Mutex** – exclusive lock with priority-inheritance support to avoid priority inversion.

Monitoring & Tracing

- **DTrace** – dynamic tracing framework providing kernel-level observability via a command-line interface.

Virtualization & Containers (brief)

- **Namespaces** – isolate process identifiers, network stacks, and filesystems.
- **cgroups** – control resource allocation (CPU, memory, I/O) per group of processes.

These notes integrate storage, memory, scheduling, security, linking, and driver concepts, building on the access-matrix and capability ideas introduced earlier. They provide a

compact reference for the wide range of topics covered in section 13.

Program Threats & Malware

Trojan horse – a seemingly benign program that, when executed, misuses the executing user's privileges.

- Often placed in a directory that appears early in \$PATH.
- Variants include login emulators and spyware that install hidden daemons.

Trap door (backdoor) – a hidden mechanism inserted by a developer that bypasses normal security checks.

- May grant unrestricted access when a special user ID or password is presented.

Logic bomb – dormant code that activates under a specific condition (date, account removal, config change).

Stack / buffer overflow – writes more data than a fixed-size buffer can hold, overwriting adjacent memory (e.g., the saved return address) and hijacking control flow.

- Mitigations: safe string functions, NX/DEP bits, stack canaries, ASLR.

Virus – self-replicating code that attaches to legitimate programs and spreads to other executables.

- Categories: file, boot, macro, source-code, polymorphic, encrypted, stealth, tunneling, multipartite, armored.

Worm – a self-replicating process that spreads across a network without attaching to a host program.

- Classic example: **Morris worm** (1988) used rsh, finger buffer-overflow, and sendmail exploits, plus password-cracking.

Port scanning – probes a host’s TCP/UDP ports to discover open services; useful for reconnaissance but not an exploit itself.

Denial-of-Service (DoS) / DDoS – resource-exhaustion or protocol-abuse attacks that render a service unavailable.

Spyware – silently installs a daemon that reports usage data or downloads ads; can act as a covert channel.

Stealth virus – intercepts system calls (e.g., read) to hide infected code from scanners.

Threat	Primary Mechanism	Typical Countermeasure
Trojan horse	Misleading name/path	Verify executable origin; least-privilege execution
Trap door	Hidden bypass code	Code review; integrity checking
Logic bomb	Conditional trigger	Auditing; integrity monitoring
Buffer overflow	Overrun fixed buffer	Bounds checking; NX, canaries
Virus	Self-replication	Antivirus, code signing
Worm	Network propagation	Patch services; IDS/IPS
Port scan	Information gathering	Firewalls, stealth-scan detection
DoS/DDoS	Resource exhaustion	Rate limiting, redundancy

Cryptography & Secure Communication

Symmetric Encryption

- **Block ciphers** (DES, 3DES, AES) encrypt fixed-size blocks; combined with modes (CBC, CTR, GCM) to protect longer messages.

- **Stream ciphers** (RC4) generate a keystream XORed with plaintext; historically used in WEP, now considered insecure.

Asymmetric Encryption

RSA – security relies on the difficulty of factoring large integers.

- **Key generation** (small-number example):
 - $p = 7$, $q = 13 \rightarrow N = 91$, $\phi(N) = 72$
 - Choose $k_e = 5$, compute $k_d = 29$ because $5 \times 29 \equiv 1 \pmod{72}$.
 - Encryption: $c = m^{k_e} \pmod{N}$, Decryption: $m = c^{k_d} \pmod{N}$.

SSL/TLS Handshake (simplified)

1. **ClientHello** – client sends nonce n_c and cipher list.
2. **ServerHello** – server replies with nonce n_s and its certificate (contains $k_{e,s}$).
3. **Certificate verification** – client validates the certificate chain.
4. **Pre-master secret** – client encrypts a fresh pms with $k_{e,s}$ and sends it.
5. **Master secret** – both sides compute $ms = \text{PRF}(pms, n_c || n_s)$.
6. **Session keys** – derived from ms for symmetric encryption and MAC.

Properties achieved – confidentiality (only client/server know pms), integrity (MAC), authentication (certificate), optional forward secrecy (Diffie-Hellman variants).

Authentication & Authorization

Password Security

- Stored as **salted hashes**: $\text{salt} || H(\text{salt} || \text{password})$.
- **Salting** prevents pre-computed dictionary attacks.
- Modern systems enforce longer passwords, use strong hash algorithms (SHA-256+), and restrict read access to the password file.

One-Time Passwords (OTP)

- **Challenge-response**: system generates random challenge *ch*, user computes $H(\text{pw}, \text{ch})$ and sends it.
- Implementations: hardware tokens, software calculators, PIN-based generators, code-books, S/Key (hash chain).

Two-Factor Authentication (2FA)

- Combines **something you know** (password/PIN) with **something you have** (OTP token).

Biometrics

- **Fingerprint**, **palm**, **iris**, etc., provide “something you are.”
- Typically combined with a password for stronger 2FA.

Windows 7 Security Model

- **Access token** – created at log-on; contains **User SID**, **Group SIDs**, **Privilege list**, and **Integrity label** (untrusted, low, medium, high, system).
- **Security descriptor** attached to each object:
 - **Owner SID**, **DACL** (Access-Allowed/Denied entries), **SACL** (audit).
- **Mandatory Integrity Control (MIC)** enforces *no-write-up*: a lower-integrity process cannot write to a higher-integrity object.
- **User Account Control (UAC)** – admin accounts receive a **standard token** (medium) and an **elevated token** (high); elevation requires user consent.

Component	Role
Access token	Carries identity, groups, privileges, integrity
SID	Unique identifier for users/groups
DACL	Discretionary ACL – who may access the object
SACL	Audit ACL – what actions generate logs
MIC	Mandatory integrity – prevents low-integrity writes to high-integrity objects

RBAC & Privileges (Solaris 10 example)

- **Roles** aggregate **privileges** (system-call rights).

- Users activate a role to obtain the necessary privileges for a specific task, implementing least-privilege.

Security Policies & “Security Through Obscurity”

- **Policy** – documented set of rules defining permissible actions.
 - **Security through obscurity** – relying on hidden implementation details rather than robust controls; generally discouraged.
-

Filesystem Structures & Storage

Directory Organization

- Directories are **acyclic graphs** linking names to inodes; hard links create multiple paths to the same inode, forming a general **graph**.

IA-32 Segmentation (Intel)

- **Segment table** entry contains a **base address** and a **limit** (size).
- **Logical address** = segment selector + offset; the hardware adds the base and checks that $\text{offset} \leq \text{limit}$.
- Provides memory protection and modular address spaces.

Block Allocation Strategies

Strategy	Description
Direct blocks	Inode stores direct pointers to data blocks.
Single-indirect	Inode points to a block that holds an array of data-block numbers.
Double-indirect	Two levels of indirection for very large files.
Indexed allocation	Dedicated index block lists data blocks in order.
Contiguous allocation	Files occupy consecutive blocks → high throughput, risk of external fragmentation.

Linked allocation	Each block stores the address of the next block.
-------------------	--

Ext3 Journaling (brief)

- Writes metadata updates to a **journal** before applying them to the main file system.
- On crash, committed journal entries are replayed, guaranteeing a consistent state.

Seek Time & Disk Geometry

- **Seek time** = time for the read/write head to move to the target track.
- Combined with **rotational latency** to give total access time.

SSDs & SATA

- **Solid-state disks** use flash memory; no moving parts, lower latency, wear-leveling needed.
- **SATA** (Serial ATA) replaces older ATA parallel bus, providing higher transfer rates.

File-System Metadata

- **Superblock** – global file-system information (size, block size, inode count).
- **Inode** – per-file metadata (owner, permissions, timestamps, block pointers).



Scheduling Algorithms

Algorithm	Category	Key Property
FCFS (First-Come-First-Served)	Non-preemptive	Simple, can cause long average turnaround
RR (Round-Robin)	Preemptive	Fixed quantum; fair sharing
SJF (Shortest-Job-First)	Non-preemptive	Minimizes average waiting time (requires knowledge of burst length)
SRPT (Shortest-Remaining-Processing-Time)	Preemptive	Optimal for minimizing average response time

Priority scheduling	Preemptive / Non-preemptive	Executes higher-priority tasks first; can cause starvation
Multilevel Queue	Hybrid	Separate queues for interactive vs. batch jobs
Feedback (aging)	Adaptive	Demotes a task after it consumes its quantum, preventing starvation
EDF (Earliest-Deadline-First)	Real-time	Guarantees deadline satisfaction if total utilization ≤ 1
Rate-Monotonic	Real-time	Fixed priority based on task period; optimal for static-priority systems
SSTF (Shortest-Seek-Time-First) – disk scheduler	I/O	Reduces average seek distance
C-SCAN / LOOK – disk scheduler	I/O	Provides more uniform wait times than simple SCAN
CFQ (Completely Fair Queuing) – block I/O	I/O	Gives each process a fair share of disk bandwidth
Lottery	Probabilistic	Tickets determine probability of being scheduled

- **CFS (Completely Fair Scheduler)** – Linux’s default CPU scheduler; assigns each runnable task a *weight* derived from its nice value and distributes CPU time proportionally.

Memory Management

Segmentation & Paging

- **Segmentation** divides a process’s address space into logical segments (code, data, stack) with base/limit checks.

- **Paging** breaks memory into fixed-size pages; a page table maps virtual page numbers to physical frames.

Translation Lookaside Buffer (TLB)

- **TLB** caches recent page-table translations.
- **TLB miss** triggers a page-table walk; **TLB reach** = total memory covered by the TLB.

Buddy Allocator

- Manages free memory in power-of-two sized blocks; adjacent *buddies* are merged when both are free.

Slab Allocator

- Provides caches of pre-allocated objects of the same size (e.g., `task_struct`).
- Each cache consists of **slabs** (one or more contiguous pages).

Swap Space & Demand Paging

- Pages not in RAM are written to a **swap area**; on a page fault the needed page is read back.
- **Demand paging** loads pages only when accessed; **pre-paging** can load a set of pages proactively.

Page-Replacement Policies

Policy	Approximation	Typical Use
FIFO	Simple queue	Academic examples
LRU (approx.)	Clock / second-chance	General purpose
LFU	Frequency counters	Workloads with stable hot pages
Optimal	Future knowledge (theoretical)	Benchmarking
Working-set	Tracks recent references	Reduces thrashing

- **Thrashing** occurs when frequent page faults consume most CPU time, often mitigated by adjusting the working-set size or using a **page-fault-frequency** strategy.
-

Concurrency & Synchronization

Primitive Overview

Primitive	Typical Use	Notes
Spinlock	Short critical sections on SMP	Busy-wait; disables preemption on uniprocessor
Semaphore	Longer waits; resource counting	Can block the caller
Mutex	Exclusive lock with priority inheritance	Prevents priority inversion
Reader-Writer lock	Many readers, few writers	Slim RW lock in Linux reduces overhead
Monitor (condition variables)	Encapsulates lock + queue	Implemented with pthread_cond_wait/signal

Classic Problems

- **Deadlock** – circular wait, hold-and-wait, no preemption, and mutual exclusion.
- **Starvation** – a process never obtains needed resources (often due to priority scheduling).
- **Priority inversion** – lower-priority task holds a lock needed by a higher-priority task; mitigated by priority inheritance.

Semaphore Example (from earlier notes)

- Binary semaphore implements a **mutex**; counting semaphore tracks multiple identical resources.

Network & I/O

Socket API

- **TCP socket** – reliable, ordered byte stream; uses sequence numbers, acknowledgments, retransmission.

- **UDP socket** – connectionless datagram service; no ordering or reliability guarantees.

Port Scanning & Firewalls

- **Port scanning** (e.g., nmap) discovers open services; firewalls can block scans or log suspicious patterns.
- **Stateful inspection** tracks connection state to allow only legitimate return traffic.

IDS / IPS

Approach	Basis
Signature-based	Known byte patterns of attacks
Anomaly-based	Deviations from learned normal behavior

DoS / DDoS Mitigations

- Rate limiting, traffic filtering, redundant server farms, and upstream scrubbing.

SSL/TLS Integration

- Uses **asymmetric** key exchange to agree on a symmetric session key, then protects the session with **symmetric encryption** and MACs.
-

Software Protection Mechanisms

Tripwire (Integrity Checking)

- Computes cryptographic hashes of protected files and stores them in a tamper-resistant database.
- Periodic scans report added, removed, or altered files.

Sandboxing & Secure Execution

- **Capability-based** or **language-based** protection (e.g., Java's stack inspection, CAP's software capabilities) limits what code can do.

Buffer-Overflow Defenses

- Compiler options (-fstack-protector), **NX/DEP**, **ASLR**, and safe library functions (strncpy).

Virus Classification (recap)

- File, boot, macro, polymorphic, encrypted, stealth, tunneling, multipartite, armored.
-



System Components & Process Model

Task Control Block (TCB)

- Holds process state: **PID**, **PPID**, credentials (UID/GID/effective IDs), **register set**, **open-file table**, **signal handlers**, **memory map**, **scheduling info**.

Shared Memory & IPC

- **Shared memory segment** – mapped into multiple processes; provides fast communication but requires explicit synchronization (semaphores, mutexes).
- **Pipes** – unidirectional byte streams with kernel-managed buffers.

VFS (Virtual File System)

- Uniform API (open, read, write) abstracts underlying file-system implementations (ext3, NFS, procfs).
- Core objects: **inode**, **dentry**, **superblock**, **file**.

Loadable Kernel Modules (LKMs)

- Inserted with insmod; register with the kernel (e.g., register_chrdev).
- Conflict resolution ensures exclusive use of hardware resources.

Shared Libraries

- **Dynamic linking** loads .so files at run-time; code is position-independent (PIC) and shared across processes, reducing memory footprint.
-



Storage Hardware & Performance

Device	Characteristics
--------	-----------------

HDD (magnetic)	Rotational latency, seek time, sequential transfer rates
SSD (flash)	No moving parts, low latency, wear-leveling, higher random I/O performance
SATA	Serial ATA interface; higher bandwidth than legacy ATA
SCSI	Parallel (or Serial Attached SCSI) bus; supports multiple devices and high throughput
DMA	Allows peripherals to read/write memory without CPU intervention
Cache hierarchy	L1/L2/L3 caches reduce memory access latency; dirty bits track modified cache lines

- **Seek time** = time to move the read/write head; combined with **rotational latency** to compute overall I/O latency.



Security & Policy Concepts

- **Security through obscurity** – relying on hidden implementation details; not a substitute for robust controls.
- **Social engineering** – manipulating humans (phishing, dumpster diving) to bypass technical safeguards.
- **DMCA** – anti-circumvention law that restricts bypassing of technological protection measures.
- **Security policies** – documented rules (e.g., “all external-facing apps must undergo code review”) that map to technical controls (firewalls, RBAC, audit logging).

These notes integrate threat models, cryptographic foundations, authentication mechanisms, Windows 7 protection primitives, filesystem internals, scheduling strategies, memory management techniques, concurrency controls, network services, and hardware considerations, providing a cohesive reference for advanced operating-system and security study.